CSE 405
Worked Final-Exam Solutions

Sec A · 87 worked entries · six academic sessions

Contents

How to use this guide 5

Navigation by slide deck and exact topic 6

Numerical and conceptual checkpoints 8

Introduction 9

A002 · 2023–24 · Q1(b) . . . 9
A021 · 2022–23 · Q1(a) . . . 10
A025 · 2022–23 · Q2(b) . . . 11
A043 · 2021–22 · Q1(a) . . . 12
A046 · 2021–22 · Q2(a) . . . 13
A066 · 2021–22 · Q8(a) . . . 14
A070 · 2020–21 · Q1(b) . . . 15
A071 · 2020–21 · Q1(c) . . . 16
A072 · 2020–21 · Q2(a) . . . 17
A073 · 2020–21 · Q2(b) . . . 18

Physical Security 19

A044 · 2021–22 · Q1(b) . . . 19
A128 · 2018–19 · Q6(b) . . . 20
A129 · 2018–19 · Q7(a) . . . 21
A130 · 2018–19 · Q7(b) . . . 22
A131 · 2018–19 · Q8(a) . . . 24
A132 · 2018–19 · Q8(b) . . . 25
A134 · 2018–19 · Q9(b) . . . 26

Operating Systems Security 27

A053 · 2021–22 · Q4(a) . . . 27
A074 · 2020–21 · Q2(c) . . . 28

A075 · 2020–21 · Q3(a) 29

Buffer Overflow **30**

A001 · 2023–24 · Q1(a) 30
A005 · 2023–24 · Q2(b) 32
A083 · 2020–21 · Q5(c) 33
A085 · 2020–21 · Q6(c) 34
A086 · 2020–21 · Q7(c) 35
A087 · 2020–21 · Q8(a) 36
A090 · 2019–20 · Q1(b) 37
A133 · 2018–19 · Q9(a) 38
A135 · 2018–19 · Q10 39

Filesystem Security **40**

A097 · 2019–20 · Q3(c) 40

Malware **41**

A008 · 2023–24 · Q3(b) 41
A028 · 2022–23 · Q3(b) 42
A029 · 2022–23 · Q3(c) 43
A033 · 2022–23 · Q4(d) 44
A049 · 2021–22 · Q3(a) 45
A057 · 2021–22 · Q5(a) 46
A059 · 2021–22 · Q5(c) 47
A069 · 2020–21 · Q1(a) 48
A077 · 2020–21 · Q3(c) 49
A079 · 2020–21 · Q4(b) 50
A092 · 2019–20 · Q2(a) 51
A095 · 2019–20 · Q3(a) 52
A096 · 2019–20 · Q3(b) 53
A112 · 2018–19 · Q1(c) 54
A114 · 2018–19 · Q2(a) 55
A115 · 2018–19 · Q2(b) 56
A116 · 2018–19 · Q2(c) 57
A117 · 2018–19 · Q2(d) 58
A118 · 2018–19 · Q3(a) 59
A121 · 2018–19 · Q3(d) 60

Network Security **61**

A004 · 2023–24 · Q2(a) . . . 61
A006 · 2023–24 · Q2(c) . . . 62
A010 · 2023–24 · Q4(a) . . . 63
A022 · 2022–23 · Q1(b) . . . 64
A023 · 2022–23 · Q1(c) . . . 65
A027 · 2022–23 · Q3(a) . . . 66
A060 · 2021–22 · Q6(a) . . . 67
A063 · 2021–22 · Q7(a) . . . 68
A078 · 2020–21 · Q4(a) . . . 69
A099 · 2019–20 · Q4(b) . . . 70
A100 · 2019–20 · Q4(c) . . . 71
A119 · 2018–19 · Q3(b) . . . 72
A120 · 2018–19 · Q3(c) . . . 73

DNS Security **74**
A009 · 2023–24 · Q3(c) . . . 74
A030 · 2022–23 · Q4(a) . . . 76
A061 · 2021–22 · Q6(b) . . . 77
A098 · 2019–20 · Q4(a) . . . 78
A122 · 2018–19 · Q4(a) . . . 79
A123 · 2018–19 · Q4(b) . . . 80

Firewalls, Tunneling and IDS **81**
A007 · 2023–24 · Q3(a) . . . 81
A011 · 2023–24 · Q4(b) . . . 82
A026 · 2022–23 · Q2(c) . . . 83
A031 · 2022–23 · Q4(b) . . . 84
A032 · 2022–23 · Q4(c) . . . 85
A058 · 2021–22 · Q5(b) . . . 86
A062 · 2021–22 · Q6(c) . . . 87
A067 · 2021–22 · Q8(b) . . . 88
A068 · 2021–22 · Q8(c) . . . 89
A076 · 2020–21 · Q3(b) . . . 90
A080 · 2020–21 · Q4(c) . . . 91
A094 · 2019–20 · Q2(c) . . . 92
A101 · 2019–20 · Q5(a) . . . 93
A102 · 2019–20 · Q5(b) . . . 94
A103 · 2019–20 · Q5(c) . . . 95

A111 · 2018–19 · Q1(b) 96
A124 · 2018–19 · Q4(c) 97
A125 · 2018–19 · Q4(d) 98

Review checklist **99**

Sources and section scope **100**

How to use this guide

87 Sec A worked entries across six academic sessions: 84 whole-question entries and 3 entries limited to Sec A components. Both printed sections of each yearly paper are checked; assignment is by the current topic split. The standalone crypto, number-theory, password-guessing/recovery and web-attack sets are excluded. Each entry appears once under its primary teaching deck, with links from every relevant Sec A topic. Exam tips precede the reasoning.

Question summaries are paraphrases, not official transcriptions or a marking scheme. Consult the linked physical paper page for original code/diagrams. Every indexed Sec A task has a worked answer, necessary assumptions, explanation and an exam tip. Mixed entries identify the omitted Sec B tasks and distinguish printed marks from marks for the retained components. Related/extension flags identify tasks beyond the local slides rather than silently pretending they are fully taught.

Notation and limits: toy memory examples use the paper's stated 32-bit conventions; actual compiler/ABI layouts can differ. Probabilities identify their assumptions. A network address is not an authenticated identity. Several "guaranteed attack" requests are under-specified; the answers explain the intended mechanism and the conditions needed for it.

The PDF uses XeLaTeX typesetting; this HTML embeds native MathML and diagrams for offline reading in a browser supporting MathML. The Markdown is editable and renders dollar math on GitHub. All sources, analysis and build tools stay within the section layout.

Navigation by slide deck and exact topic

Introduction — Ch01-Introduction.pdf

CIA, attacker models and security scope: A043 · 2021–22 Q1(a); A046 · 2021–22 Q2(a); A070 · 2020–21 Q1(b)

Security principles and scenario analysis: A002 · 2023–24 Q1(b); A021 · 2022–23 Q1(a); A071 · 2020–21 Q1(c); A073 · 2020–21 Q2(b); A087 · 2020–21 Q8(a)

Access matrices, ACLs, capabilities and RBAC: A025 · 2022–23 Q2(b); A066 · 2021–22 Q8(a); A072 · 2020–21 Q2(a)

Physical Security — Ch02-Physical-Security.pdf

Pin-tumbler locks and master-key extensions: A128 · 2018–19 Q6(b); A129 · 2018–19 Q7(a); A130 · 2018–19 Q7(b)

Biometrics, authentication factors, SIM and RFID: A044 · 2021–22 Q1(b); A073 · 2020–21 Q2(b); A131 · 2018–19 Q8(a); A134 · 2018–19 Q9(b)

Faraday shielding and physical information leakage: A132 · 2018–19 Q8(b)

Operating Systems Security — Ch03-OS-Security.pdf

OS trust, paging, boot and persistent memory images: A053 · 2021–22 Q4(a); A071 · 2020–21 Q1(c); A074 · 2020–21 Q2(c); A075 · 2020–21 Q3(a); A094 · 2019–20 Q2(c); A102 · 2019–20 Q5(b); A132 · 2018–19 Q8(b)

Buffer Overflow — Ch03-BufferOverflow.pdf

Stack/heap corruption and buffer-overflow payload reasoning: A001 · 2023–24 Q1(a); A005 · 2023–24 Q2(b); A075 · 2020–21 Q3(a); A087 · 2020–21 Q8(a); A090 · 2019–20 Q1(b); A135 · 2018–19 Q10

StackGuard, ASLR and memory-corruption defenses: A005 · 2023–24 Q2(b); A083 · 2020–21 Q5(c); A086 · 2020–21 Q7(c); A134 · 2018–19 Q9(b)

Return-to-libc and calling conventions: A083 · 2020–21 Q5(c); A085 · 2020–21 Q6(c)

Format-string reads and writes: A133 · 2018–19 Q9(a)

Filesystem Security — Ch03-FilesystemSecurity.pdf

Unix permissions, setuid and filesystem access: A087 · 2020–21 Q8(a); A097 · 2019–20 Q3(c)

Malware — Ch04-Malware.pdf

Backdoors, logic bombs and insider abuse: A112 · 2018–19 Q1(c); A118 · 2018–19 Q3(a)

Encrypted, polymorphic and metamorphic viruses: A057 · 2021–22 Q5(a); A096 · 2019–20 Q3(b); A114 · 2018–19 Q2(a)

Worm propagation, epidemic models and case studies: A008 · 2023–24 Q3(b); A033 · 2022–23 Q4(d); A069 · 2020–21 Q1(a); A077 · 2020–21 Q3(c); A095 · 2019–20 Q3(a); A117 · 2018–19 Q2(d)

Botnet creation, C&C and malicious agents: A059 · 2021–22 Q5(c); A079 · 2020–21 Q4(b); A092 · 2019–20 Q2(a); A121 · 2018–19 Q3(d)

Rootkit cloaking, hooking and DLL injection: A028 · 2022–23 Q3(b); A049 · 2021–22 Q3(a); A116 · 2018–19 Q2(c)

Adware, spyware and malware economics: A029 · 2022–23 Q3(c); A057 · 2021–22

Q5(a); A095 · 2019–20 Q3(a); A115 · 2018–19 Q2(b)

Network Security — Ch05-NetworkSecurity.pdf

ARP spoofing, MITM and detection: A004 · 2023–24 Q2(a); A078 · 2020–21 Q4(a); A119 · 2018–19 Q3(b); A124 · 2018–19 Q4(c)

Switch MAC-table flooding and port security: A027 · 2022–23 Q3(a); A099 · 2019–20 Q4(b)

DHCP starvation, spoofing and defenses: A004 · 2023–24 Q2(a); A023 · 2022–23 Q1(c)

TCP hijacking, spoofed segments and optimistic ACKs: A006 · 2023–24 Q2(c); A007 · 2023–24 Q3(a); A010 · 2023–24 Q4(a); A060 · 2021–22 Q6(a); A100 · 2019–20 Q4(c); A119 · 2018–19 Q3(b); A120 · 2018–19 Q3(c)

DoS/DDoS, reflection and resource exhaustion: A007 · 2023–24 Q3(a); A010 · 2023–24 Q4(a); A070 · 2020–21 Q1(b); A092 · 2019–20 Q2(a); A094 · 2019–20 Q2(c)

SYN flooding and stateless SYN cookies: A007 · 2023–24 Q3(a); A022 · 2022–23 Q1(b); A063 · 2021–22 Q7(a)

DNS Security — Ch06-1-NetworkDNS.pdf

DNS poisoning, Kaminsky races, pharming and DNSSEC: A007 · 2023–24 Q3(a); A009 · 2023–24 Q3(c); A030 · 2022–23 Q4(a); A061 · 2021–22 Q6(b); A098 · 2019–20 Q4(a); A122 · 2018–19 Q4(a); A123 · 2018–19 Q4(b)

Firewalls, Tunneling and IDS — Ch06-2-FW-Tunneling-IDS-REDUCED.pdf

Reconnaissance, fingerprinting and vulnerability assessment: A026 · 2022–23 Q2(c); A067 · 2021–22 Q8(b); A076 · 2020–21 Q3(b)

Firewall state, evasion and defense limits: A007 · 2023–24 Q3(a); A011 · 2023–24 Q4(b); A032 · 2022–23 Q4(c); A068 · 2021–22 Q8(c); A079 · 2020–21 Q4(b)

VPN and IPsec AH/ESP encapsulation: A058 · 2021–22 Q5(b); A111 · 2018–19 Q1(b)

Intrusion detection, base rates and audit data: A080 · 2020–21 Q4(c); A094 · 2019–20 Q2(c); A101 · 2019–20 Q5(a); A102 · 2019–20 Q5(b); A124 · 2018–19 Q4(c)

Honeypot topology, monitoring and overhead: A031 · 2022–23 Q4(b); A059 · 2021–22 Q5(c); A062 · 2021–22 Q6(c); A103 · 2019–20 Q5(c); A125 · 2018–19 Q4(d)

Numerical and conceptual checkpoints

Exercise	Checkpoint
Three independently mastered pin chambers	$2^3 = 8$ total theoretical profiles, seven additional
Common master lock	Bottom pins (2,2,5,2,6); wafers (2,2,0,3,0)
SI worm model	$I(t) = N/[1 + ((N - I_0)/I_0)e^{-\beta N t}]$
IDS example: prevalence 0.1%, sensitivity 99%, FPR 1%	About 9.02% of alarms are true positives
Toy printf cursor model	secret1 slot 6; controlled first input word slot 13
Toy alphabet-only return	0x41424349 encodes as little-endian ICBA; clean EBP restoration is a separate constraint
Known-salt password	Salt is public; it prevents shared precomputation, not guessing a single account

Introduction

Primary deck: Ch01-Introduction.pdf

A002 · 2023–24 · Q1(b)

Match five scenarios to security principles · 10 marks

Original: past-paper PDF p. 2. Sitting date: 06 September 2025. Printed Section A.

Topics: Security principles and scenario analysis.

> **Question summary:** Match principles to a valet key, bypassable laptop screen lock, irreversible water-damage sticker, bank recovery questions overridden by a representative, and encryption at rest and in transit.

Exam tip. Give the principle followed by the causal link to the example, rather than a list of principle names.

Scenario	Principle and explanation
Valet key opens ignition/doors but not trunk	**Least privilege:** grant the driver the subset of access required for parking.
Specialized equipment bypasses screen lock	**Threat model and work factor:** a screen lock addresses some casual-access threats; physical/offline attackers require stronger protections. Compare attack cost with asset value. Full-disk encryption and boot trust address additional paths.
Sticker remains red after water exposure	**Compromise recording:** preserve evidence of a violation when prevention is impractical. It detects the condition; it does not prevent liquid damage or prove intent.
Representative waives a strong recovery answer	**Human factors and complete mediation:** a human fallback becomes the weakest authentication path. A strong secret is ineffective if an unverified story bypasses it. Use a consistent, independently verified recovery process.
Encrypt internal/external traffic and stored data	**Defense in depth:** protect different exposure paths and trust boundaries. Access control, key separation and monitoring remain necessary if an endpoint is compromised.

Shannon’s maxim means assuming the adversary knows the system; none of these scenarios is primarily solved by hiding its design. Several principles can fit, but explain the concrete failure or protection.

A021 · 2022–23 · Q1(a)

Least privilege versus separation; why record compromise? · 10 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Security principles and scenario analysis.

> **Question summary:** Differentiate least privilege and separation of privilege, and explain the importance of compromise recording with examples.

Exam tip. Use two distinct examples; do not describe both principles merely as "restrict access."

Least privilege minimizes the rights of each subject/process. A photo editor receives access to selected pictures rather than every user file; a service runs without administrator rights.

Separation of privilege/responsibility requires multiple independent conditions or parties for a sensitive operation. Releasing a payment may require approvals from both an initiator and a separate reviewer. Giving one administrator a smaller permission set is least privilege; requiring two approvals is separation.

Compromise recording captures evidence of violations: protected authentication/access logs, change journals, IDS events and physical surveillance. Perfect prevention is rarely achievable. Records support timely detection, reconstruction, containment, accountability and recovery. A record is useful only if it has trustworthy timestamps, integrity protection, appropriate retention and someone responsible for responding. Forwarding logs to a protected collector limits deletion by a compromised host.

A025 · 2022–23 · Q2(b)

Why owner-only access cannot replace access-control systems · 12 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Access matrices, ACLs, capabilities and RBAC.

> **Question summary:** Give at least two scenarios in which users other than a file owner need access, refuting a proposal to restrict all file permissions to owners only.

Exam tip. State which operation the non-owner needs and which operation must remain forbidden.

1. **Shared project:** several collaborators need to read/write a common report or repository while other users must be excluded. Owner-only access would prevent collaboration or encourage unsafe account/password sharing.
2. **System executables and libraries:** administrator-owned tools must be readable/executable by ordinary users while modifications remain administrator-only. Making every user an owner would undermine integrity.
3. **Backup and service accounts:** a backup worker may need read-only access across selected directories; a web service may need to read published files but not edit them.

An access matrix represents subject/object rights. Per-object ACLs express the column compactly; roles/groups reduce repeated administration. Grant only required operations, deny unnecessary writes and separate ownership/administration from usage. Permission checks must use the requesting identity and honor revocation.

A043 · 2021–22 · Q1(a)

Information, IT and cybersecurity scope diagram · 5 marks

Original: past-paper PDF p. 15. Sitting date: 30 October 2023. Printed Section A.

Topics: CIA, attacker models and security scope.

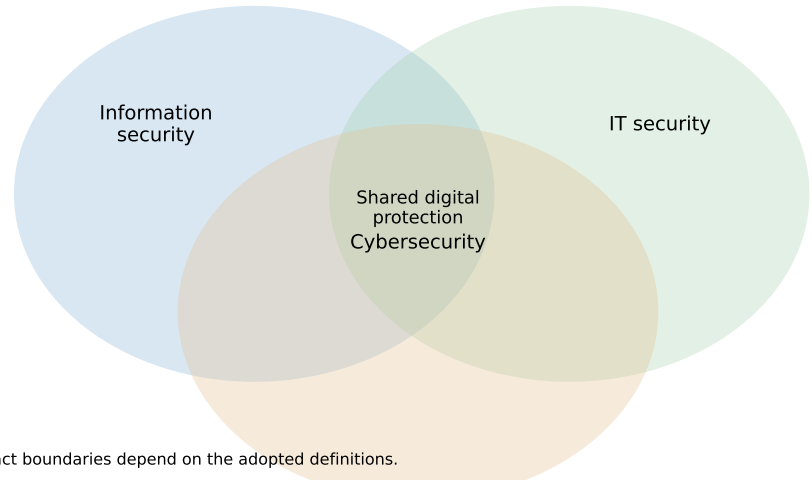
> **Question summary:** Illustrate the relationship between information security, cybersecurity and IT security with a Venn diagram.

Coverage: related application/extension. The introduction defines CIA and security, but does not specify this three-set taxonomy.

Exam tip. Put examples in the regions. Do not claim one taxonomy is mathematically mandatory when definitions vary.

Use an explicitly stated convention: **information security** protects information in all forms (paper, speech and digital); **IT security** protects computing systems and the data/services they process; **cybersecurity** protects activities/assets exposed through cyberspace and interconnected systems.

These fields overlap strongly but are not universally defined as three strict nested sets. A useful conceptual Venn diagram puts digital information protection in the common intersection. Paper-file confidentiality is in information security outside the digital overlap; network-service disruption and connected physical-device safety illustrate cybersecurity concerns beyond protecting information alone.



Conceptual overlap; exact boundaries depend on the adopted definitions.

The deck's CIA framework applies throughout the overlap. Label this as a conceptual convention, since the supplied slides do not define a unique formal set relationship.

A046 · 2021–22 · Q2(a)

Classify attacks by the information-security CIA triad · 5 marks

Original: past-paper PDF p. 15. Sitting date: 30 October 2023. Printed Section A.

Topics: CIA, attacker models and security scope.

> **Question summary:** Categorize security attacks with respect to confidentiality, integrity and availability.

Exam tip. Give a concrete attack for each goal and identify the property it violates.

Use the **information-security** CIA triad:

Goal	Violation	Example
Confidentiality	Unauthorized disclosure	Eavesdropping on a private conversation or reading a protected file.
Integrity	Unauthorized modification or fabrication	Altering a transaction, corrupting a file or injecting forged records.
Availability	Denial or degradation of timely authorized service	Flooding a server, exhausting its connection state or deliberately dropping traffic.

One attack can violate several goals: malware can steal documents, change them and disable access. Classify its particular effects rather than assuming every attack has exactly one category. Authentication helps establish who is acting, but **A here means availability**, as in Sec A's Introduction PDF p. 2.

A066 · 2021–22 · Q8(a)

Delegation through GRANT OPTION and revocation dependencies · 12 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: Access matrices, ACLs, capabilities and RBAC.

> **Question summary:** Analyze GRANT SELECT ON employees TO Alice WITH GRANT OPTION and revoke access safely when Alice leaves.

Coverage: related application/extension. Access-control delegation is an application of matrices/ACLs; DBMS-specific grant/revoke syntax is not taught in WebDB-security.

Exam tip. Revoke the delegation chain as well as the departing user's own privilege, and name the assumed SQL dialect.

The statement lets Alice read `employees` and **delegate SELECT** to other principals. Read-only access to her own session is therefore not the whole authority: she can create additional readers, including an account she retains after departure.

On departure, remove Alice's direct SELECT and grant option, disable her authentication, terminate active sessions where needed and audit every grant derived from her authority. In SQL systems supporting dependency-aware cascading revocation, the intended form is:

```
REVOKE SELECT ON employees FROM Alice CASCADE;
```

Exact syntax/behavior is DBMS-dependent. PostgreSQL supports cascading dependent grants; MySQL has different privilege/revocation semantics and does not accept this generic clause as a portable solution. Inspect the actual grant graph and remove derived rights using the engine's mechanisms. Another independent grant to a recipient may survive revocation of Alice's grant path.

Revocation cannot erase previously copied data. Minimize exposure in advance: avoid unnecessary grant option, use managed roles, limit sensitive columns/rows and review grants.

PostgreSQL REVOKE semantics.

A070 · 2020–21 · Q1(b)

Selective packet dropping against a competitor · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: CIA, attacker models and security scope; DoS/DDoS, reflection and resource exhaustion.

> **Question summary:** An ISP deliberately drops 25% of a competing provider's VoIP packets traversing its routers. Identify the attack.

Exam tip. Availability includes timely usable service. A 25% degradation can be a DoS attack without a total outage.

This is a **selective denial-of-service/degradation attack** by an on-path network operator. It violates availability by making the competitor's voice calls unreliable even if the service is not completely unavailable. The operator abuses a trusted forwarding position; the attacker need not impersonate an endpoint or decrypt the voice content.

Packet loss produces gaps, jitter/retransmissions where applicable and poor call quality. Encrypting VoIP can protect content and integrity but cannot force a malicious router to forward packets. Detection compares loss/quality across paths and traffic classes; mitigation can include independent paths/providers and routing around the hostile infrastructure.

A071 · 2020–21 · Q1(c)

Why compromise recording matters and which tools support it · 11 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Security principles and scenario analysis; OS trust, paging, boot and persistent memory images.

> **Question summary:** Explain the importance of compromise recording and name tools used for it.

Exam tip. A log that nobody checks or an attacker can silently erase is weak compromise recording.

Prevention can fail, so systems need evidence that a violation occurred. Compromise records support detection, incident reconstruction, identification of affected accounts/assets, containment and recovery. They can also deter abuse and support accountability.

Examples include OS security/event logs, authentication and privilege-change auditing, application/database access logs, file-integrity monitoring, network flow/packet metadata, IDS alerts and physical surveillance. A protected central log collector helps preserve evidence when an individual host is compromised.

Record reliable timestamps, actor/subject, action, target, result and source context. Protect log integrity/access, monitor collector health, retain relevant records and connect alerts to a response process. Logging everything indiscriminately can create privacy, storage and analysis problems; select evidence according to the threat model.

A072 · 2020–21 · Q2(a)

RBAC diagram, benefits and limitations · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Access matrices, ACLs, capabilities and RBAC.

> **Question summary:** Define role-based access control with a figure, benefits and drawbacks.

Exam tip. Separate user-to-role assignment from role-to-permission assignment in the diagram.

RBAC associates permissions with organizational **roles**, then assigns users to roles. A user's permitted actions come from their active role memberships rather than a separately edited list for every object.

```
Alice --+--> Accountant --> read ledger, enter transactions
Bob ----+
Carol ------> Auditor ----> read ledger, view audit trail
```

Roles can have hierarchies; constraints can enforce separation of duties, such as preventing one person from both initiating and approving a payment.

Benefits: simpler onboarding/offboarding, consistent least-privilege job profiles, reusable policy and more understandable audits. Limitations: role explosion when exceptions proliferate, stale memberships after job changes, conflicts between accumulated roles and difficulty expressing fine-grained context such as time, location or a particular patient's care relationship.

A role name alone is not a security control: authenticate the user, authorize each operation and review role assignments. Deny unauthorized actions by default.

A073 · 2020–21 · Q2(b)

Mismatched boarding-pass and identity checks · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Security principles and scenario analysis; Biometrics, authentication factors, SIM and RFID.

> **Question summary:** Explain how online-printable boarding passes permit a no-fly-list bypass in the described airport checks, and fix the weakness.

Coverage: related application/extension. An application of complete mediation/authentication; the exact airport workflow is in the paper.

Exam tip. Show what each checkpoint sees and why two individually plausible documents are not sufficient together.

The described checks fail to bind **the same person, reservation and boarding pass** across all stages. A listed traveler could book under an unlisted identity. At the ID checkpoint, an altered paper pass bearing the traveler's real name might satisfy the local name comparison if the checkpoint does not validate the reservation. At boarding, the traveler presents the genuine pass for the unlisted booking if the gate checks the reservation but not the person's ID.

This is an illustrative failure of **complete mediation and consistent identity binding**. It depends on the precise split checks given in the problem; it is not a statement about every airport's procedures.

Authenticate the pass/barcode against the reservation system; bind it to the verified traveler and current screening result; recheck list eligibility and identity at the stages that authorize travel; prevent substitution/reuse after checking. A digital signature on the pass protects its fields only if the scanner verifies it and ties it to the person presenting it.

Physical Security

Primary deck: Ch02-Physical-Security.pdf

A044 · 2021–22 · Q1(b)

Biometric requirements, recognition pipeline and privacy · 10 marks

Original: past-paper PDF p. 15. Sitting date: 30 October 2023. Printed Section A.

Topics: Biometrics, authentication factors, SIM and RFID.

> **Question summary:** State biometric identification requirements, describe the process, and discuss privacy-preserving challenges.

Exam tip. Explain both the matching error tradeoff and the permanence/revocation problem.

A usable biometric should have **universality** (most people possess it), **distinctiveness**, **permanence** and **collectability**. Practical criteria include accuracy/performance, user acceptability and resistance to circumvention.

```
enrollment: sensor -> quality/liveness checks -> feature extraction -> protected template
recognition: new sample -> features -> compare with template(s) -> threshold decision
```

Identification searches a population (**1:N**); verification checks a claimed identity (**1:1**). Noise means matching is approximate. Thresholds trade false accepts against false rejects; false enrollment and spoofing are separate risks.

Privacy is difficult because biometrics are persistent identifiers: a stolen fingerprint is hard to revoke, templates can enable cross-database linkage, and samples may reveal more personal information than necessary. Protect templates with access controls/encryption, minimize retention, separate databases, prefer local matching where feasible and use cancellable/template-protection methods with careful threat analysis. Plainly hashing a noisy sample does not solve approximate matching. Liveness checks and another factor reduce spoofing risk but do not make a biometric secret.

A128 · 2018–19 · Q6(b)

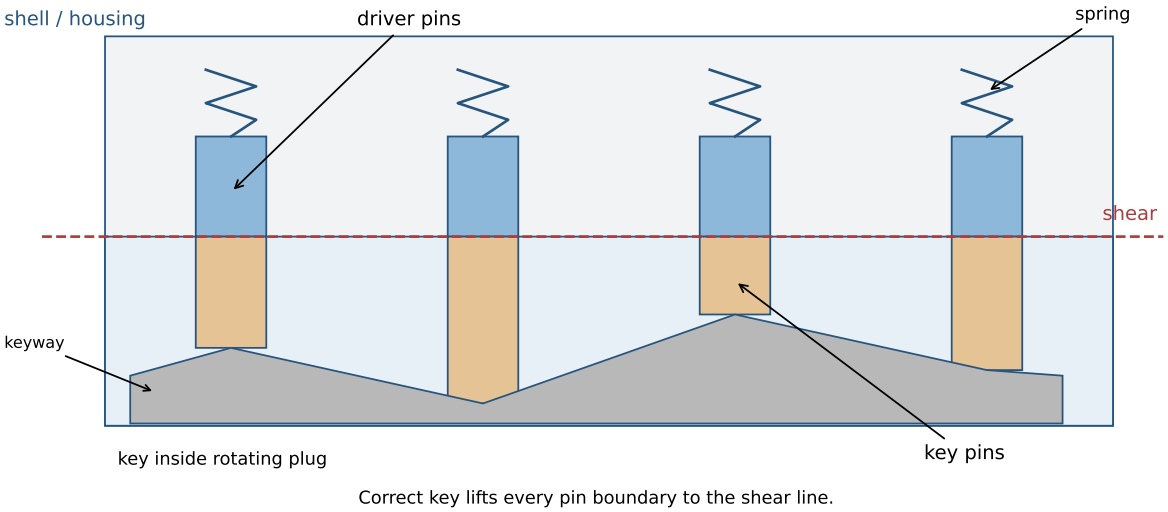
Pin-tumbler cross section · 7 marks

Original: past-paper PDF p. 27. Sitting date: 24 October 2019. Printed Section B.

Topics: Pin-tumbler locks and master-key extensions.

> **Question summary:** Draw a pin-tumbler lock cross section and label its parts.

Exam tip. The pins do not vanish or move sideways; rotation is possible because no solid pin crosses the shear line.



Correct key lifts every pin boundary to the shear line.

The fixed **housing/shell** surrounds a rotatable **plug** containing the keyway. Each chamber has a spring, a driver pin above and a key pin below. With no key or a wrong key, a solid pin crosses the shell/plug **shear line**, blocking rotation.

The correct key lifts each stack until its key-pin/driver-pin boundary lies at the shear line. The plug then rotates with the key and actuates the locking mechanism. Label springs, driver pins, key pins, key, plug, shell and shear line in the diagram.

A129 · 2018–19 · Q7(a)

Count additional keys after three master wafers · 3 marks

Original: past-paper PDF p. 27. Sitting date: 24 October 2019. Printed Section B.

Topics: Pin-tumbler locks and master-key extensions.

> **Question summary:** A five-pin lock has driver pins of height 5 and key pins 2,4,5,2, 6. Add one master wafer in each of three selected chambers. How many additional keys theoretically open it?

Coverage: related application/extension. The deck teaches basic pin-tumbler operation; master-wafer combinatorics is an extension.

Exam tip. Distinguish eight total keys from seven additional keys.

Assume the three selected chambers each gain **two distinct usable shear boundaries**, and the resulting cut choices are independent and physically valid. Each contributes two possible cut heights; unchanged chambers contribute one.

$$\text{total theoretical key profiles} = 2^3 = 8.$$

The original working key is one profile, so the number of **additional** theoretical keys is

$$8 - 1 = \boxed{7}.$$

Driver-pin height and the original key-pin values are not needed for this count. Real bitting restrictions, wafer dimensions or coincident/unusable boundaries can reduce it; the question says theoretically.

A130 · 2018–19 · Q7(b)

Construct a lock accepting both stated keys · 12 marks

Original: past-paper PDF p. 27. Sitting date: 24 October 2019. Printed Section B.

Topics: Pin-tumbler locks and master-key extensions.

> **Question summary:** Given L1 key-pin heights (2,4,5,2,6) and L2 (4,2,5,5,6), driver heights 5, design L3 with master wafers so both keys work, and show K1 opening it.

Coverage: related application/extension. Master-key construction extends the basic lock slides; the geometric interpretation is stated explicitly.

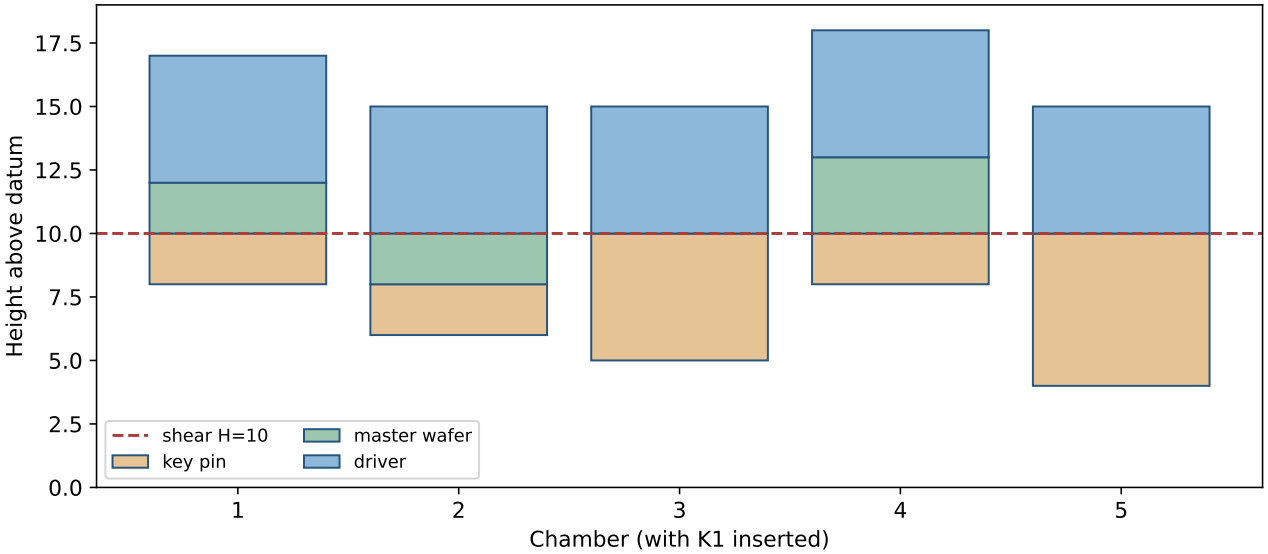
Exam tip. Use a shorter bottom pin plus the difference wafer; two original pin lengths placed in series do not create the intended boundaries.

Use a common key datum/shear height H and interpret the supplied numbers as the key-pin lengths required by the two key profiles. In each chamber take the **smaller key pin** and a wafer equal to the absolute difference:

Chamber	L1 pin	L2 pin	L3 key pin	Master wafer	Driver
1	2	4	2	2	5
2	4	2	2	2	5
3	5	5	5	none	5
4	2	5	2	3	5
5	6	6	6	none	5

For a concrete sketch choose $H = 10$. K1 lifts the stacks by $(8, 6, 5, 8, 4)$ units, since lift equals H minus its corresponding original key-pin length. In chamber 1 the top of the 2-unit key pin is at H (wafer above); in chamber 2 the top of key pin plus wafer is at H; chambers 3/5 have their ordinary boundary at H; chamber 4 uses the top of its 2-unit pin with wafer above. Thus no solid pin crosses the shear line.

K2 uses lifts $(6, 8, 5, 5, 4)$, selecting the alternative boundary in chambers 1,2,4. Keep adequate spring/chamber clearance for the chosen dimensions.



Three independently mastered chambers theoretically also admit six unintended mixed profiles besides the two named keys: $2^3 - 2 = 6$.

A131 · 2018–19 · Q8(a)

Good biometric criteria · 6 marks

Original: past-paper PDF p. 28. Sitting date: 24 October 2019. Printed Section B.

Topics: Biometrics, authentication factors, SIM and RFID.

> **Question summary:** Name essential and desirable characteristics of biometric authentication.

Exam tip. Give a short explanation of each property; “secure and fast” is too vague.

The usual core criteria are **universality**, **distinctiveness/uniqueness**, **permanence** and **collectability**. A useful implementation also needs good **performance** (speed, accuracy and robustness), **acceptability** to users and resistance to **circumvention** such as spoofed samples.

In practice these are degrees rather than absolutes: some people lack a measurable feature, samples change with age/injury, and environmental noise affects capture. Provide alternatives and evaluate false-accept/false-reject rates rather than promising perfect identity recognition.

A132 · 2018–19 · Q8(b)

Faraday cage and OS impersonation-token short notes · 9 printed marks (mixed question)

Original: past-paper PDF p. 28. Sitting date: 24 October 2019. Printed Section B.

Topics: Faraday shielding and physical information leakage; OS trust, paging, boot and persistent memory images.

> **Question summary:** From the printed "any three" short notes, cover the Sec A options: (i) Faraday cage and (iii) impersonation token.

Sec A scope: Covers only options (i) Faraday cage and (iii) OS impersonation token, 3 marks each. Options (ii) blind SQL injection and (iv) homograph attack remain in Sec B. The printed 9 marks require any three choices.

Coverage: related application/extension. The printed subquestion is 9 marks for any three choices. This guide covers only two Sec A choices (3 marks each); it is not a full 9-mark answer. OS impersonation-token mechanics extend the supplied privilege material.

Exam tip. Keep shielding under physical security and impersonation under OS identity/privilege. The printed question's remaining two options belong to Sec B and are excluded from this answer.

(i) Faraday cage. A conductive enclosure attenuates electromagnetic coupling between its interior and exterior. Charge redistribution and induced currents help shield against external fields and reduce information leakage from electronic emanations. In computer security, shielding can protect equipment or reduce unintended radio emissions. Its effectiveness depends on frequency, material, seams, openings and construction; it does not guarantee perfect isolation at every frequency or block unrelated acoustic leakage.

(iii) Impersonation token. In an operating-system security context, a token represents a principal's security context, such as identity, groups and privileges. A server thread can temporarily impersonate a client so that access checks use the client's rights rather than the server's broader authority. This supports least privilege when the token and permitted impersonation level are properly checked. Token theft or excessive impersonation rights can permit unauthorized access. Limit the scope, protect token handles and reliably restore the original context after the operation.

Microsoft access-token documentation describes these identity and privilege contexts.

The OS-token explanation extends the supplied process-privilege material. It is distinct from a web session cookie and is not a SQL-injection mechanism.

A134 · 2018–19 · Q9(b)

Expand ICCID, IMSI and ASLR · 3 marks

Original: past-paper PDF p. 28. Sitting date: 24 October 2019. Printed Section B.

Topics: Biometrics, authentication factors, SIM and RFID; StackGuard, ASLR and memory-corruption defenses.

> **Question summary:** Write the full forms of ICCID, IMSI and ASLR.

Coverage: related application/extension. SIM identifiers are in the physical deck/notes; ASLR is a memory-defense extension.

Exam tip. Write “Identity” for IMSI and “Randomization” for ASLR.

- **ICCID:** Integrated Circuit Card Identifier.
- **IMSI:** International Mobile Subscriber Identity.
- **ASLR:** Address Space Layout Randomization.

ICCID identifies the card/profile; IMSI identifies the mobile subscription. Neither should be confused with a phone number. ASLR randomizes memory locations to make address prediction harder; it does not correct an unsafe memory access.

Operating Systems Security

Primary deck: Ch03-OS-Security.pdf

A053 · 2021–22 · Q4(a)

Secrets persisted in swap/hibernation · 5 marks

Original: past-paper PDF p. 16. Sitting date: 30 October 2023. Printed Section A.

Topics: OS trust, paging, boot and persistent memory images.

> **Question summary:** Explain how virtual memory or a swap partition can be an attacker target.

Exam tip. Connect the RAM secret to its disk copy; “virtual memory is fake memory” is not an explanation.

Paging can write memory pages to disk; hibernation writes a much larger memory snapshot. Passwords, keys, documents and session tokens that were temporarily present in RAM can therefore remain in swap/hibernation storage after an application exits or the screen locks.

An attacker with physical disk access, an alternate boot path or sufficient privileges may extract those remnants, bypassing the original OS’s access checks. Virtual memory itself is not a vulnerability: the exposure is sensitive state persisted without suitable protection.

Use disk/swap encryption with appropriate key handling, protect boot and physical access, restrict dump/image files and minimize secret lifetime. An encrypted disk whose key is accessible to an attacker in a live unlocked system has a different threat model.

A074 · 2020–21 · Q2(c)

Forensic preservation: master and working images, custody · 11 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: OS trust, paging, boot and persistent memory images.

> **Question summary:** Why make two copies of digital evidence? State precautions and how to maintain chain of custody.

Coverage: related application/extension. OS/hibernation notes mention forensic investigation; full acquisition and custody procedures are supplementary.

Exam tip. Explain what each copy is for and how you establish that it faithfully represents the acquired evidence.

Keep one verified **preservation/master image** and use a separate **working copy** for examination. Analysis can alter metadata, recovery state or software behavior; a preserved baseline allows repeatable checking. Preserve the original device as well, when applicable. Merely possessing two unverified copies proves nothing.

Acquire a suitable bit-stream image using validated tools and a write blocker where appropriate. Record device identifiers, acquisition time, tool/version, operator and relevant system state. Hash the acquired image with a suitable cryptographic hash and verify copied images against it. Decide separately how to capture volatile evidence before power changes; imaging a live system may itself change data and needs documented handling.

Maintain a custody record for every transfer: evidence ID, who released/received it, timestamps, purpose, storage/location and integrity checks. Store originals/master images with restricted access and tamper-evident handling. A hash establishes consistency, while custody documentation establishes the handling history; neither substitutes for the other.

Technical reference: NIST SP 800-86.

A075 · 2020–21 · Q3(a)

Why preserve memory-region boundaries? · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: OS trust, paging, boot and persistent memory images; Stack/heap corruption and buffer-overflow payload reasoning.

> **Question summary:** Explain the risks of mixing heap and stack storage and give an example manipulation.

Exam tip. Distinguish address-space sharing from unsafe overlap or lack of bounds enforcement.

Stack storage follows call/return lifetimes; heap allocations have explicit allocator-managed lifetimes. Both usually occupy one process's **virtual address space** already. The security goal is valid bounds, lifetimes and suitable protections—not a false claim that separate labeled regions prevent all overflow.

If an unchecked heap allocation/overflow can reach live stack control data, an attacker can corrupt local state, saved frame pointers or return addresses. Conversely, an unbounded stack allocation can collide with heap mappings or other regions if guards/limits are absent. For example, copying an oversized request into a heap buffer adjacent to a control structure can overwrite a callback pointer; if storage is mixed with a stack frame, the analogous overwrite may replace the saved return address.

Region separation and guard pages reduce accidental collisions and help enforce stack/heap limits. They are not complete isolation within the process: an arbitrary write can still target either region. Use bounds/lifetime-safe code, guarded mappings, allocation checks and control-flow defenses.

Buffer Overflow

Primary deck: Ch03-BufferOverflow.pdf

A001 · 2023–24 · Q1(a)

Trace the actual unsafe copies and construct a conditional control-flow payload · 10 marks

Original: past-paper PDF p. 1. Sitting date: 06 September 2025. Printed Section A.

Topics: Stack/heap corruption and buffer-overflow payload reasoning.

> **Question summary:** For the printed 32-bit little-endian C program, decide whether hack() can be invoked and give symbolic-address payload pseudocode. All defenses are disabled; main reads 300 bytes into str[30]; overflow_in_action has buffer[300], two heap pointers, strcpy calls and an indirect callback.

Coverage: related application/extension. The source does not fix frame offsets or heap allocator layout. A conditional symbolic payload is justified; an unconditional offset is not.

Exam tip. Mark the destination and bound for every write. A stack overflow and a heap overflow have different reachable targets.

Find all three writes before choosing an attack. `fread(str,1,300,...)` already overflows main's 30-byte stack array. It does not append a NUL. `strcpy(buffer,str)` may also exceed 300 bytes if no earlier terminator exists. Finally, `strcpy(data->name,str)` overruns the 30-byte heap allocation for sufficiently long strings.

A robust exam answer uses the first overflow to replace **main's saved return address**, provided execution survives the intervening copies/callback and returns normally. Define S as the address of `str[0]`, R as the address of main's saved return word, H as the address of `hack`, and $d = R - S$. Derive d from the actual frame; declaration order does not specify padding/alignment.

```
from struct import pack
# S, R, H come from the particular exam/debugger stack layout.
d = R - S
assert 30 <= d and d + 5 <= 300
payload = b'A' * d + pack('<I', H) + b'\0'
```

The terminating zero bounds subsequent copies **only if** the packed address contains no earlier zero; it does not undo their memory corruption. This payload establishes the intended overwritten return word, but success is conditional on heap layout, valid pointer variables, allocation metadata and successful return from `overflow_in_action`. A chain may need to preserve intervening words. If `hack` returns, a valid continuation is also needed.

Do not invent a heap adjacency guarantee. `ptr` is allocated before `data`; an ordinary forward overflow from `data->name` does not necessarily overwrite the earlier callback allocation. Also `ptr->fp=random_print` is assigned after the stack copy. The printed C plus "all defenses disabled" does not determine a universal byte-exact successful payload. State the layout/survival assumptions instead of claiming that 30 heap bytes automatically reach `fp`.

Fix: read at most `sizeof str-1` bytes, terminate explicitly, validate lengths before both copies, and eliminate unnecessary mutable callback/heap state.

A005 · 2023–24 · Q2(b)

Leaked stack address and one-byte saved-frame-pointer overwrite · 10 marks

Original: past-paper PDF p. 3. Sitting date: 06 September 2025. Printed Section A.

Topics: Stack/heap corruption and buffer-overflow payload reasoning; StackGuard, ASLR and memory-corruption defenses.

> **Question summary:** 32-bit x86 with ASLR: main prints simple[32] address and reads with fgets(simple,32). hack copies nine input bytes backward into buf[8]. Construct inputs for a 31-byte shellcode, identifying necessary stack-layout assumptions.

Coverage: related application/extension. Compiler local ordering, padding and the location of a surviving fake frame are not specified, so the intended off-by-one exploit is conditional.

Exam tip. A leaked address bypasses secrecy, not bounds checks or frame-layout constraints. Show both the one-byte overwrite and the later pivot.

The first input can put the **31-byte shellcode** in `simple`; `fgets(...,32,...)` reads at most 31 bytes and appends NUL. The printed address S reveals its location despite ASLR. Shellcode bytes must be suitable for this input path (in particular, avoid an early newline). Executable stack is assumed because other defenses are disabled.

The second read fills `input[9]` exactly. The loop's first write is `buf[8]=input[8]`, one byte beyond `buf`. The intended exploit is an **off-by-one saved-EBP overwrite**. This requires `buf+8` to be the low byte of saved EBP; compiler padding or a different local ordering defeats that premise. The source alone does not establish this layout.

With a `leave; ret` epilogue, corrupting hack's saved EBP changes the EBP restored in main. When main later executes `leave`, it pivots ESP to a chosen address F. The word at F is treated as saved EBP and the word at $F+4$ as the return address. Place S at $F+4$ and arrange a usable continuation. Because only one byte is changed, F must share the original saved frame pointer's upper 24 bits; a low-byte wrap cannot change the other bytes.

One possible **conditional layout** puts a fake two-word frame in the eight controlled bytes of `buf`, sets the ninth byte to the low byte of its address, and returns into `simple`. It additionally requires that this memory survive main's epilogue and that saved EBP has the required upper bytes. Alternatively place a fake frame in another controlled area satisfying the same constraints. A 31-byte shellcode already consumes almost all of `simple`; do not silently also assume eight extra writable frame bytes fit there.

Under that intended layout, define F as the address of `buf[0]` and M as main's original frame pointer. The second input has the concrete symbolic form:

```
from struct import pack
assert (F & 0xffffff00) == (M & 0xffffff00)
second_input = pack('<I', 0x41414141) + pack('<I', S) + bytes([F & 0xff])
# 8 bytes form the fake frame in buf; byte 9 alters saved EBP.
```

The first input is the stipulated 31-byte shellcode itself; `fgets` supplies its terminating NUL. The fake frame's first word becomes EBP after main's `leave`, while its second word transfers to S. A shellcode that does not return avoids needing a valid later stack continuation. Without the assumed adjacency/address relation this byte sequence is not guaranteed to work.

A083 · 2020–21 · Q5(c)

Four defenses against return-to-libc · 10 marks

Original: past-paper PDF p. 19. Sitting date: 25 September 2022. Printed Section B.

Topics: Return-to-libc and calling conventions; StackGuard, ASLR and memory-corruption defenses.

> **Question summary:** Briefly explain four countermeasures against return-to-libc attacks.

Coverage: related application/extension. Buffer-overflow basics are taught; code reuse and several defense mechanisms require supplementary material.

Exam tip. State what each defense interrupts and one relevant limitation; do not count "NX" as if it prevents code reuse by itself.

1. **Remove the memory-corruption primitive:** bounded operations, correct allocation/lifetime checks and memory-safe implementation prevent the overwrite used to redirect control.
2. **Stack canaries:** verify a protected value before returning to detect many contiguous overwrites of saved control data. Leaks and noncontiguous writes can bypass this protection.
3. **ASLR with PIE and randomized libraries:** make target code/data addresses hard to predict. Information leaks and weak entropy reduce its effectiveness.
4. **Control-flow/return protection:** a protected shadow stack checks return addresses; suitable CFI constrains indirect transfers. These directly limit forged return/call chains, subject to the mechanism's coverage.

A non-executable stack alone is **not** a sufficient return-to-libc defense: the attack reuses existing executable code. W^X/NX still reduces code-injection options and belongs in layered protection.

A085 · 2020–21 · Q6(c)

Place system arguments using 32-bit cdecl · 10 marks

Original: past-paper PDF p. 20. Sitting date: 25 September 2022. Printed Section B.

Topics: Return-to-libc and calling conventions.

> **Question summary:** Using diagrams, determine placement of the system() argument in a return-to-libc attack.

Coverage: related application/extension. The supplied C/stack material is background; the cdecl libc-chain layout is an extension.

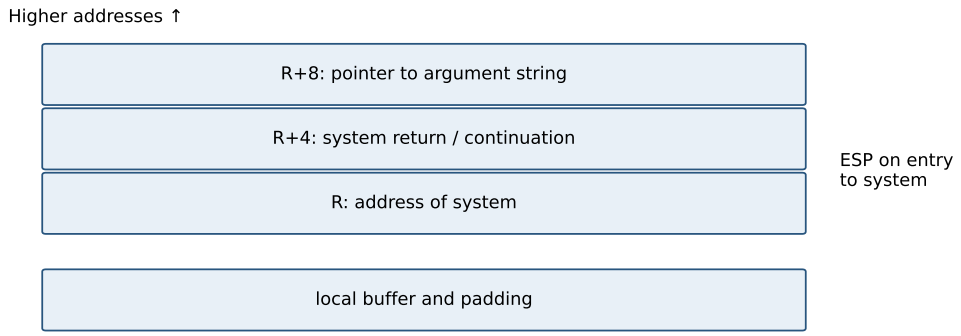
Exam tip. Simulate ESP after each `ret`; that removes the common off-by-four mistake.

Assume the paper's conventional **32-bit x86 cdecl** frame and a vulnerable function whose saved return address can be overwritten. Let R denote that saved return slot, S the address of `system`, E a usable continuation such as `exit`, and B a valid pointer to the NUL-terminated command string in this toy layout.

```
higher addresses
R+8   B       first argument to system
R+4   E       system's apparent return address
R     S       vulnerable function's overwritten return address
... padding/local buffer ...
lower addresses
```

When the vulnerable function executes `ret`, it loads EIP from R and ESP becomes $R+4$. On entry, `system` expects its return address at `[ESP]` and its first argument at `[ESP+4]`, so the argument belongs at **R+8**, not immediately after the overwritten return word.

If `system` returns, a safe continuation needs its own correctly arranged arguments/frame; simply writing an `exit` address does not guarantee a clean exit status argument. Addresses, offset to R, input NUL constraints, ASLR and canary state must be established for the exact program.



This arrangement does not carry over unchanged to x86-64 ABIs, which generally pass initial arguments in registers.

A086 · 2020–21 · Q7(c)

StackGuard check and scope · 10 marks

Original: past-paper PDF p. 20. Sitting date: 25 September 2022. Printed Section B.

Topics: StackGuard, ASLR and memory-corruption defenses.

> **Question summary:** Define StackGuard and explain with a diagram how it counters buffer overflow.

Exam tip. Put the check before control transfer and state which overwrite path crosses the canary.

StackGuard is a compiler-inserted stack-canary defense. A protected word is placed between vulnerable stack storage and saved control data in a function frame. Before returning, generated code checks that the canary still equals its expected value; a mismatch aborts rather than using a corrupted return address.

```
higher addresses: caller state / return address
                  saved frame pointer
                  CANARY  <-- checked before return
                  local arrays / other locals
lower addresses
```

A conventional sequential overflow from a local buffer toward the return address must cross the canary. Random canaries make an unseen correct replacement difficult; some historical schemes use terminator values to obstruct string copying. Exact frame ordering depends on implementation.

This **detects some corruption**, not the unsafe write itself. A canary leak, an overwrite that skips the canary, corruption of non-control data or a heap attack may avoid the check. A process may also be harmed before reaching its epilogue. Combine it with bounds-safe code, ASLR, NX and control-flow protection.

A087 · 2020–21 · Q8(a)

Audit scanf, privileged file opening and a name-based access race · 15 marks

Original: past-paper PDF p. 20. Sitting date: 25 September 2022. Printed Section B.

Topics: Stack/heap corruption and buffer-overflow payload reasoning; Unix permissions, setuid and filesystem access; Security principles and scenario analysis.

> **Question summary:** Analyze attacks on the printed C program (code on p. 21): scanf into buffer[60], fopen(fn,"a+"), access(fn,W_OK), fwrite; state environmental assumptions.

Coverage: related application/extension. Unsafe C, Unix privileges and complete-mediation principles support the answer; the exact race analysis is an application.

Exam tip. Trace object identity through the descriptor. This example is an open-then-check race, not the usual check-then-open ordering.

Unbounded input: `scanf("%s",buffer)` has no width, so an input exceeding the 60-byte array (including its terminator) can corrupt neighboring locals or saved control state. A specific control-flow attack depends on frame layout, address knowledge, protections and process privileges. Use a bounded read, check its result and handle long input deliberately.

Privilege/identity mismatch: if this is a setuid program, `fopen` uses effective-identity permissions, whereas `access` ordinarily checks the real identity. Opening before the authorization decision may already create a file or otherwise have effects; the program should not acquire unauthorized capabilities and then rely on a later pathname check.

Name race, with the printed order preserved: the code opens `fn` **before** `access`. In a writable directory an attacker can arrange for the name to resolve to a protected target during `fopen`, then replace the name with an allowed file before `access`. The check succeeds for the new name, but `fp` still refers to the originally opened target, so `fwrite` affects that object. This requires a useful effective privilege and attacker control of the pathname/directory (plus applicable symlink protections). A literal fixed relative pathname is not automatically attacker-controlled.

Other defects include unchecked `fopen` failure and insufficient handling of I/O errors. `strlen` also requires a valid terminated input string.

Repair: avoid setuid where possible; drop privileges before opening; use a trusted directory/descriptor-based access strategy, constrain symlink traversal as appropriate, and operate on the verified opened object. A separate earlier `access` followed by `fopen` would still be race-prone.

A090 · 2019–20 · Q1(b)

Heartbleed as an out-of-bounds read · 12 marks

Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Stack/heap corruption and buffer-overflow payload reasoning.

> **Question summary:** Define Heartbleed and explain its major consequences.

Coverage: related application/extension. The buffer deck teaches memory-bound errors; the particular OpenSSL case is supplementary.

Exam tip. Name the trust error: using the peer's claimed length instead of validating it against the actual received buffer.

Heartbleed (CVE-2014-0160) was a missing-bounds-check bug in affected OpenSSL TLS/DTLS heartbeat handling. A peer supplied a small payload but claimed a larger length; the implementation copied the claimed number of bytes from process memory into its reply without checking that those bytes belonged to the received payload.

This is a **buffer over-read/information disclosure**, not intrinsically an overwrite or a break of TLS mathematics. A request could expose up to roughly 64 KiB of nearby memory, and repetition could expose more. Possible leaked values included passwords, session tokens, application data and private-key material present in that process; any particular secret's presence was not guaranteed.

Patch affected software and restart it so vulnerable processes are gone; then replace potentially exposed keys/certificates and reset affected credentials/sessions as appropriate. Replacing a certificate without fixing the vulnerable process can expose the new key again. Network encryption does not protect secrets the endpoint itself mistakenly returns.

Original disclosure.

A133 · 2018–19 · Q9(a)

Four format-string tasks using the stated stack-word cursor · 12 marks

Original: past-paper PDF p. 28. Sitting date: 24 October 2019. Printed Section B.

Topics: Format-string reads and writes.

> **Question summary:** For gets(user_input); printf(user_input), construct inputs to print secret1, change secret1, print secret2 and print secret3. Cursor starts 0xaabbcc20; secret1=0xaabbcc34, secret2=0xaabbcc38, secret3=0x11223344, input=0xaabbcc50; integers are 4 bytes.

Coverage: related application/extension. The slides teach unsafe C but do not develop format-string exploitation; the paper supplies a special cursor model.

Exam tip. `%x` consumes a value; `%s` and `%n` consume a pointer. Keep those roles separate.

Adopt the paper's toy convention that the first variadic word consumed by a conversion is at **0xaabbcc20**, with 4-byte progression. Real `printf` has no portable exposed "internal pointer," so these offsets are an explicit exam model.

The word index (1-based) for address a is

$$j = 1 + \frac{a - \texttt{0xaabbcc20}}{4}.$$

Thus secret1's slot is 6, secret2 begins at 7, and the first controlled input word is slot 13. Assume little-endian and a `printf` dialect supporting positional conversions.

Task	Toy input bytes	Meaning
Print secret1 integer	`b'%6$x'` (or `%6$d`)	Read slot 6 as the integer, not as a pointer.
Modify secret1	`b'\x34\xcc\xbb\xaa%13$n'`	Controlled slot 13 contains its address; `%n` writes the number printed so far (4 for this literal prefix).
Print secret2 string	`b'\x38\xcc\xbb\xaa%13$.10s'`	Dereference its address, displaying at most 10 bytes.
Print secret3 string	`b'\x44\x33\x22\x11%13$.10s'`	Dereference address 0x11223344, even in a different frame; assumes those 10 bytes are readable.

For secret1, skipping five 4-byte words with five `%08x` conversions then using `%x` is the non-positional conceptual equivalent. Do **not** use `%6$n` to modify the integer: that interprets secret1's current value as an address. The supplied address must be placed in a controlled argument slot.

The raw address bytes in the Python byte literals must actually be supplied as bytes; typing the characters `\x34` at runtime does not make `printf` decode a C string escape. The paper's hint confusing C lexical escapes and runtime format parsing should not be copied literally. String termination, input newlines and implementation support must be checked.

Fix by using a constant format: `printf("%s",user_input)` and a bounded read. This removes the attacker-controlled format rather than filtering selected percent sequences.

A135 · 2018–19 · Q10

Toy stack frames and an alphabet-only return target · 15 marks

Original: past-paper PDF p. 29. Sitting date: 24 October 2019. Printed Section B.

Topics: Stack/heap corruption and buffer-overflow payload reasoning.

> **Question summary:** For the printed main/foo/bar C snippets, draw all frames using supplied addresses and create alphabet-only sample.txt so bar returns directly to main. Each source instruction is stipulated to occupy one byte; integer=4 bytes.

Coverage: related application/extension. Stack reasoning is taught, but this question imposes artificial instruction sizes/frame addresses and leaves the file-read body unspecified.

Exam tip. Show both the alphabetic return address and the invalid restored-frame caveat. A control transfer is weaker than clean continued execution.

Use the problem's **toy 32-bit frame-pointer model** (downward stack, saved EBP at EBP and return at EBP+4). It is intentionally not a realistic compiler/ISA layout. Given frame pointers are main **0xaabbccdd**, foo **0xaabbddcc**, bar **0xaaccbbdd**.

Frame	Slot	Address/value
main	saved caller EBP / return	EBP / EBP+4; caller values arbitrary as allowed
main	a, then b	EBP−4 holds 3; EBP−8 reserved for b (not yet assigned at the call)
foo	saved EBP	0xaabbddcc holds 0xaabbccdd
foo	return to main	0xaabbddd0 holds 0x41424349 under the stipulated source-statement counting
foo	p	0xaabbddd4 holds 3
foo	str[100], then c	EBP−100 through EBP−1; c at EBP−104 (not yet assigned)
bar	saved EBP	0xaaccbbdd holds 0xaabbddcc
bar	return to foo	0xaaccbbe1 holds the instruction after `bar(str)`
bar	s	0xaaccbbe5 points to foo's str at 0xaabbdd68
bar	buf[10]	0xaaccbbd3 through 0xaaccbbdc

The desired main instruction is `int b=4`, two one-byte source instructions after `int a=3`, hence $0x41424347 + 2 = 0x41424349$. Its little-endian encoding is **`ICBA`**, all English letters. Under the no-padding model, the return slot is $10 + 4 = 14$ bytes after `buf[0]`:

```
sample = b'A'*10 + b'BBBB' + b'ICBA'
```

`strcpy` writes these 18 bytes then its NUL. `bar`'s `ret` loads the chosen main address directly, skipping foo's normal continuation.

Limitations that matter: `BBBB` restores EBP to 0x42424242, not main's actual frame pointer. The alphabets-only restriction cannot encode the supplied main EBP bytes for a full clean restore; reaching the chosen instruction does not guarantee subsequent correct accesses/return. A real compiler, padding or stack protections change the payload. The body of the "necessary code to read str" is unspecified, so its length prevents a unique numeric normal return-to-foo address unless a further source-statement convention is given. The stipulated frame addresses also leave gaps; do not invent a contiguous real call-stack layout from them.

Filesystem Security

Primary deck: Ch03-FilesystemSecurity.pdf

A097 · 2019–20 · Q3(c)

Setuid/chmod under the paper's no-revert assumption · 11 marks

Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Unix permissions, setuid and filesystem access.

> **Question summary:** Alice wants Bob's photo program without exposing her confidential files; Bob wants his code unreadable to Alice. Use Unix setuid/chmod, assume the process cannot revert to real UID after adopting its setuid identity, and consider exfiltration by malicious code.

Exam tip. State the prompt's artificial assumption and separate inaccessible secrets from deliberately granted input.

Under the **paper's simplified identity rule**, Bob can own the executable, keep Alice from reading/writing it, allow her to execute it and set its setuid bit. For example `chmod 4711 photo-tool` makes it executable by others but not readable/writable by them. Its effective UID becomes Bob's when Alice runs it.

Keep Alice's confidential directories/files inaccessible to Bob (for example private directory mode 700 and files 600), while granting Bob/the program only the needed photo access through a selected group/ACL or separate shared directory. The program then cannot read Alice's private files using Bob's effective identity; no access means no contents to exfiltrate, even if it can write to a public location or network socket.

This does **not** mean arbitrary setuid code is generally safe. An application that must read a sensitive photo can also leak that photo; permissions cannot distinguish an authorized read for processing from an unauthorized use of the same read data. The process may also inherit open file descriptors from its caller, so do not pass confidential descriptors.

The prompt explicitly rules out reverting to the caller's real UID. Real Unix credential/saved-UID behavior is more complicated, and an unreadable executable is not universal source-code protection (binaries can leak implementation details). For actual deployment use an isolated account/sandbox, explicit data access and controlled output.

Malware

Primary deck: Ch04-Malware.pdf

A008 · 2023–24 · Q3(b)

Code Red patch status and propagation · 5 marks

Original: past-paper PDF p. 4. Sitting date: 06 September 2025. Printed Section A.

Topics: Worm propagation, epidemic models and case studies.

> **Question summary:** Did Code Red exploit a zero-day vulnerability? Explain its propagation.

Exam tip. Distinguish a newly observed worm from a previously unknown vulnerability. New malware can exploit an old bug.

The major July 2001 **Code Red** outbreak exploited an IIS Indexing Service buffer overflow for which Microsoft had already published a patch in June 2001. In that outbreak it was a known, unpatched vulnerability, rather than a zero-day exploit.

An infected IIS server scanned for other vulnerable servers and sent a specially crafted HTTP request. Successful exploitation ran the worm on the next server, which repeated the scanning. This automated replication produced rapid growth; it did not require each new victim to open an email attachment.

Microsoft MS01-033 documents the patched flaw; CAIDA's Code Red analysis records the July propagation.

A028 · 2022–23 · Q3(b)

Kernel API hooking and rootkit concealment · 12 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Rootkit cloaking, hooking and DLL injection.

> **Question summary:** List rootkit features and explain kernel-level hiding through function hooking.

Exam tip. Draw the intercepted query path and distinguish hooking from direct structure modification.

A rootkit provides concealment/persistence for other malicious activity. It can hide processes, files, services, registry entries, ports or user accounts. It is distinct from the payload it hides.

In a kernel hook, privileged malicious code redirects a kernel query's dispatch path or alters its result. For example:

```
process-list tool -> system query -> hooked enumeration routine
                                  -> remove malicious process record
                                  -> return a plausible filtered list
```

The process may keep executing while ordinary tools report it absent. File/directory and network enumeration can be filtered similarly. Installing a kernel hook normally needs elevated privileges or a separate privilege-escalation vulnerability. A user-mode API hook affects fewer observation paths; **DKOM** instead directly alters kernel bookkeeping structures and need not hook the query itself.

Detect with cross-view comparisons, integrity checks and trusted offline examination. A compromised kernel can lie to tools running under it, so ordinary successful antivirus output is not proof of a clean system. Recover from trusted media and rebuild when confidence cannot be restored.

A029 · 2022–23 · Q3(c)

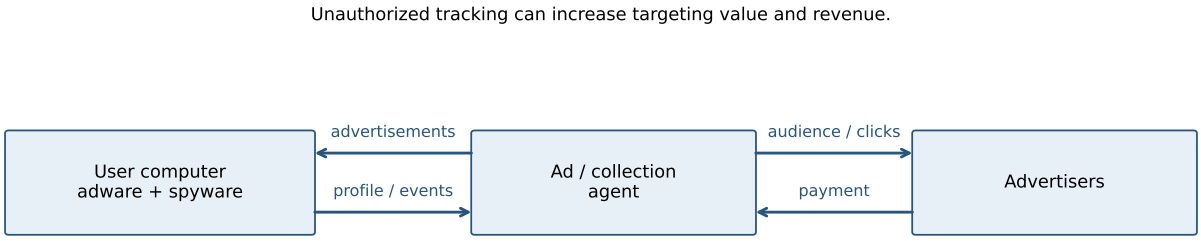
Adware data flow and spyware revenue · 11 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Adware, spyware and malware economics.

> **Question summary:** Explain adware operation with a figure and why its author benefits from adding spyware.

Exam tip. Label both the money flow and the information flow.



An installed adware component contacts an advertising agent, obtains advertisements and displays or injects them on the user's system. Advertisers pay the agent/operator for impressions, clicks or conversions.

Spyware adds collection of browsing interests, searches, application usage or other personal data. The operator can profile users, choose more profitable targeted ads and sell the data; abusive variants also steal credentials. More relevant ads can increase click/conversion revenue, explaining the incentive to combine the two components.

Keep the roles distinct: the **adware engine** displays ads, the **spyware engine** collects data, the **agent** distributes ads/receives reports, and **advertisers** fund the campaign. Not every ad-supported application secretly steals passwords; the security issue is unauthorized collection/actions and excessive privileges.

A033 · 2022–23 · Q4(d)

Zero-day meaning and detection limits · 6 marks

Original: past-paper PDF p. 12. Sitting date: 18 April 2024. Printed Section A.

Topics: Worm propagation, epidemic models and case studies.

> **Question summary:** Define a zero-day attack and explain why it is dangerous.

Exam tip. Distinguish “zero-day vulnerability,” “exploit” and “new malware signature.”

A zero-day attack exploits a vulnerability before defenders have an effective available fix or established response; the issue may be unknown to the vendor. Usage varies about discovery/disclosure, so state this timing convention.

It is dangerous because patch-based prevention may be unavailable, known signatures may not match the exploit, and a widely deployed product can expose many systems before diagnosis and mitigation. Successful exploitation can give an attacker the victim process’s privileges and access.

Least privilege, isolation, attack-surface reduction, memory-safety mechanisms, behavior monitoring and incident response can reduce impact even without a specific patch. Zero-day does not mean unstoppable; nor is every attack on an unpatched machine a zero-day if the patch already exists.

A049 · 2021–22 · Q3(a)

DLL injection and inherited process authority · 5 marks

Original: past-paper PDF p. 15. Sitting date: 30 October 2023. Printed Section A.

Topics: Rootkit cloaking, hooking and DLL injection.

> **Question summary:** Explain why DLL injection poses high security risks.

Coverage: related application/extension. Rootkit/API filtering and process hijacking are taught; detailed DLL-injection mechanics are not.

Exam tip. State whose privileges the code receives and how the trust boundary was crossed.

DLL injection causes a target process to load/execute attacker-chosen library code. The injected code runs inside that process's address space and normally with its security context. It may read secrets, intercept API calls, change behavior or conceal activity behind a trusted process name.

Injection is especially serious when a less trusted actor can influence a privileged target. Loading a DLL requires some mechanism—such as already granted debugging/write access, an unsafe loading/search path or a vulnerability. The term itself does not imply any process can inject into any other process.

Use least privilege, protected loading paths, explicit trusted library resolution, integrity controls and restrictions on cross-process modification. Rootkit API filtering is one possible consequence; injection and hooking are related but not identical.

A057 · 2021–22 · Q5(a)

Why sell many signature-distinct variants of one exploit? · 12 marks

Original: past-paper PDF p. 16. Sitting date: 30 October 2023. Printed Section B.

Topics: Encrypted, polymorphic and metamorphic viruses; Adware, spyware and malware economics.

> **Question summary:** Explain the financial benefits of creating many malware variants exploiting the same popular-software vulnerability but having different signatures.

Exam tip. Identify the reusable cost, the cheap variation and the revenue gained from longer evasion.

An exploit and the malware bytes delivered by it are different assets. Reusing one working exploit reduces development cost, while mutation/packing changes the payload's byte patterns and hashes. Customers can receive apparently distinct products without the author discovering a new vulnerability each time.

A new variant may evade a scanner that only recognizes older byte signatures, extending infection lifetime and increasing income from spam, fraud, credential theft or ransomware. Cheap automated variation also makes defenders analyze/update many samples, while the attacker amortizes the original exploit investment over a large vulnerable population.

This is an economic advantage rather than a proof of invisibility: patching the underlying flaw blocks all variants using it; behavioral detection, emulation and integrity controls can recognize shared effects. Polymorphism changes encrypted representations/decryptors; metamorphism rewrites the body while preserving behavior.

A059 · 2021–22 · Q5(c)

Disrupting C&C through coordinated investigation · 11 marks

Original: past-paper PDF p. 16. Sitting date: 30 October 2023. Printed Section B.

Topics: Botnet creation, C&C and malicious agents; Honeypot topology, monitoring and overhead.

> **Question summary:** Formulate a process to take down a botnet command-and-control server.

Exam tip. Include discovery, coordinated disruption and endpoint cleanup; a DoS attack on the suspected server is not a sound takedown plan.

Observe representative malware in a contained analysis environment or honeypot; record C&C domains/IPs, protocol, certificate/key material, connection patterns and fallback channels. Correlate evidence from multiple infections before identifying control infrastructure.

Coordinate with the responsible hosting providers, registrars, network operators and authorized response teams. Preserve evidence, arrange suspension/seizure or a controlled sinkhole of confirmed infrastructure, and prevent the replacement endpoint from becoming an attacker tool. Block validated indicators, notify affected owners and remove/rebuild infected hosts so they cannot simply reconnect elsewhere.

A centralized botnet may lose command delivery if its key server disappears. Redundant servers, domain-generation algorithms and peer-to-peer C&C can survive a single-server action; map those dependencies and plan simultaneous disruption. Taking down a C&C endpoint does not remove installed malware or recover stolen credentials.

A069 · 2020–21 · Q1(a)

WannaCry vulnerability, success and zero-day status · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Worm propagation, epidemic models and case studies.

> **Question summary:** Identify WannaCry's main vulnerability, why it succeeded and whether it was a zero-day attack.

Exam tip. Give the patch-versus-outbreak timeline, not just the word "ransomware."

WannaCry combined ransomware with worm-like propagation through vulnerable Windows **SMBv1** services, using an exploit associated with EternalBlue. The widely observed May 2017 outbreak reached systems missing the **MS17-010** update released on **14 March 2017**.

It succeeded because many exposed/reachable systems remained unpatched, legacy systems were widespread and weak segmentation allowed lateral propagation. The payload encrypted files and demanded payment; automatic spread amplified the damage.

For that outbreak, the exploited flaw was **not a zero-day**: a fix had been available before the attacks. A newly encountered ransomware executable or unavailable local signature does not make the underlying patched vulnerability unknown.

Microsoft's contemporaneous analysis, MS17-010.

A077 · 2020–21 · Q3(c)

Derive the SI epidemic worm model · 11 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Worm propagation, epidemic models and case studies.

Question summary: Explain an epidemic model of worm propagation with a figure.

Exam tip. Define the rate convention and explain all three phases of the S curve.

Let N be the fixed vulnerable population, $I(t)$ infected hosts and $S(t) = N - I(t)$ susceptible hosts. With uniform random contact and rate constant β, infections occur at rate

$$\frac{dI}{dt} = \beta IS = \beta I(N - I).$$

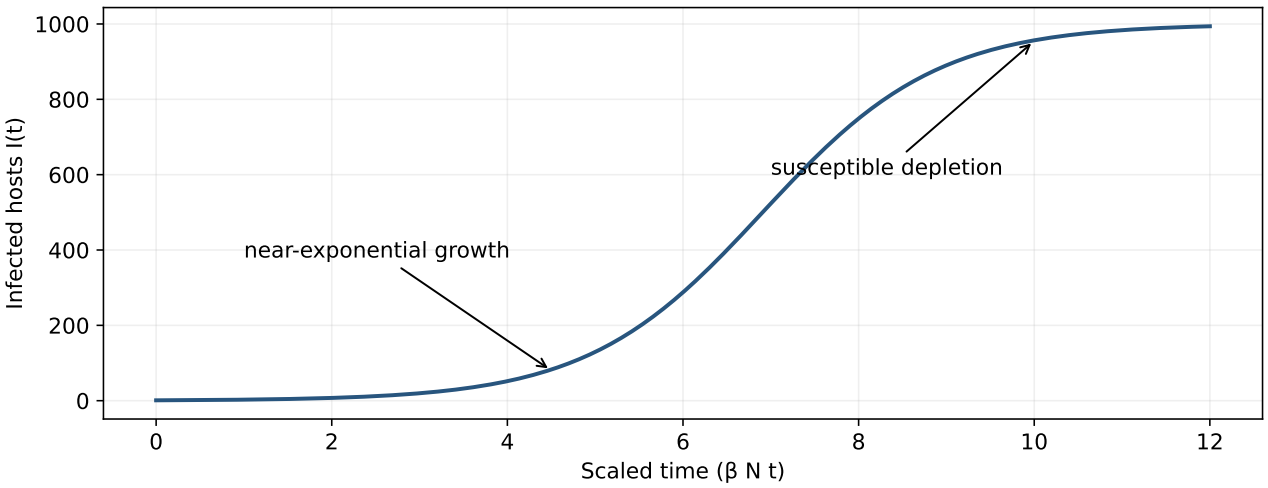
Separate variables and integrate:

$$\frac{1}{N} \log \frac{I}{N - I} = \beta t + C,$$

so for $I(0) = I_0$,

$$I(t) = \frac{N}{1 + \left(\frac{N - I_0}{I_0}\right) e^{-\beta N t}}.$$

Initially $I \ll N$ and growth is approximately exponential, $I(t) \approx I_0 e^{\beta N t}$. Around $I = N/2$, new infections grow fastest. Eventually few susceptible hosts remain and the curve saturates.



An alternative convention defines $\lambda = \beta N$ and writes $dI/dt = \lambda I(1 - I/N)$; do not confuse the units of the constants. Real networks violate uniform mixing and may have patching, recovery, scanning bias, congestion or disconnected hosts. The SI model here assumes no recovery during the modeled interval.

A079 · 2020–21 · Q4(b)

Value and defense of an internal malware agent · 12 marks

Original: past-paper PDF p. 19. Sitting date: 25 September 2022. Printed Section A.

Topics: Botnet creation, C&C and malicious agents; Firewall state, evasion and defense limits.

> **Question summary:** Explain why an attacker installs an agent inside a corporate network, delivery methods and prevention; assess whether a firewall can help.

Exam tip. Draw the outbound connection from the inside agent; that explains why "there is a firewall" is not a complete defense.

An internal agent provides persistence, local discovery, access to internal services, credential theft and a foothold for lateral movement. It can initiate an outbound C&C connection, so a perimeter policy blocking only unsolicited inbound connections may not stop it.

Delivery can involve phishing attachments, malicious installers/updates, browser/application vulnerabilities or compromised credentials used to install code. The exact route determines prevention: patching, least privilege, restricted software execution, strong authentication, segmentation and endpoint monitoring are complementary controls.

A firewall helps by limiting exposed services, enforcing internal segmentation and restricting outbound destinations/protocols. It is insufficient when malicious traffic is carried over a permitted encrypted session or authorized-looking channel, and it does not clean an already compromised endpoint. Correlate endpoint and network evidence, isolate affected systems and restore trustworthy state.

A092 · 2019–20 · Q2(a)

Botnet architecture and distributed floods · 12 marks

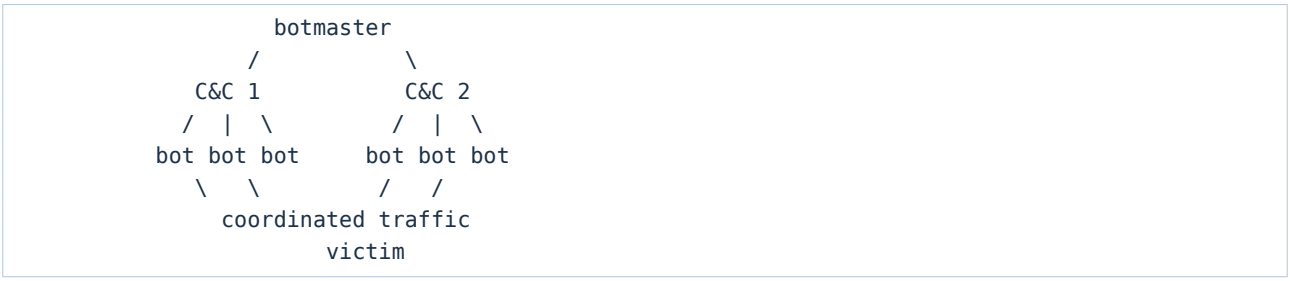
Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Botnet creation, C&C and malicious agents; DoS/DDoS, reflection and resource exhaustion.

> **Question summary:** Explain botnet components and how they launch DDoS, with a figure.

Exam tip. Separate infection/agent installation, C&C command delivery and the eventual flood payload.

A botnet is a collection of compromised hosts running remotely controlled agents. A botmaster distributes instructions via one or more C&C endpoints or a peer-to-peer overlay. Agents report status and execute commands, often without their owners' knowledge.



For DDoS, many bots simultaneously send requests/packets that exhaust the victim's link, state, CPU or application capacity. Distribution increases aggregate rate and makes simple source-IP blocking costly because legitimate users can share networks with bots. Spoofing is optional: many botnet floods use the bots' real addresses.

Defend with endpoint prevention/cleanup, segmentation, C&C disruption, rate/admission controls and upstream traffic mitigation. A perimeter filter alone cannot restore an already saturated incoming link.

A095 · 2019–20 · Q3(a)

Three reasons ransomware grew during COVID-era disruption · 12 marks

Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Adware, spyware and malware economics; Worm propagation, epidemic models and case studies.

> **Question summary:** Give three major reasons ransomware attacks increased during the COVID-19 pandemic.

Coverage: related application/extension. Malware economics is taught, but COVID-specific comparative evidence is not supplied.

Exam tip. Give three different causal mechanisms rather than three examples of phishing.

Three plausible mechanisms are:

1. **Expanded remote access:** hurried deployment of remote desktops/VPNs, unmanaged devices and weak authentication exposed more entry points.
2. **Social and operational disruption:** pandemic-themed phishing, overloaded support teams and delayed maintenance made initial compromise and persistence easier.
3. **Strong criminal incentives:** high dependence on online services and downtime-sensitive organizations increased payment pressure; reusable ransomware services/affiliates reduced attackers' development cost.

These are reasoned explanations, not a quantitative causal ranking established by the supplied slides. Connect each mechanism to the attack lifecycle: entry, propagation or monetization. Defenses follow accordingly—harden remote access, authenticate strongly, maintain patches, constrain lateral movement and keep tested isolated backups.

A096 · 2019–20 · Q3(b)

Encrypted-virus structure and detection · 12 marks

Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Encrypted, polymorphic and metamorphic viruses.

Question summary: Explain how an encrypted virus is formed and how to detect it.

Exam tip. Encrypting the body changes its bytes; the decryptor is still executable code that defenders may recognize.

A basic encrypted virus consists of a relatively fixed **decryptor/loader** plus an encrypted copy of its malicious body. At execution, the loader reconstructs the body in memory and transfers control to it. Different keys can change the ciphertext without changing the behavior.

```
on disk: [decryptor | key/derivation data | encrypted body]
run: decryptor -> plaintext body in memory -> malicious execution
```

A scanner may recognize the fixed decryptor even when the encrypted body differs. Emulation/sandbox execution can let the sample decrypt and then recognize the exposed body or behavior. Integrity baselines detect unexpected changes to legitimate files. Static heuristics may identify suspicious loading patterns but can produce false positives.

A polymorphic virus also varies its decryptor; a metamorphic virus rewrites the body itself. Emulation has time/visibility limits and can be evaded, so it is one layer rather than guaranteed detection.

A112 · 2018–19 · Q1(c)

Threshold-triggered banking-data exfiltration · 9 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Backdoors, logic bombs and insider abuse.

> **Question summary:** A programmer secretly emails account details whenever a balance exceeds USD 5,000. Identify the attack and risks.

Exam tip. Identify actor, trigger and payload separately rather than assigning a label without explaining the violation.

This is an **insider-planted malicious feature** with a condition-triggered payload. The balance threshold is a logic-bomb-style trigger; emailing private account information is a confidentiality violation. It can also be described as a Trojanized banking application because normal banking behavior conceals the malicious action. It is not necessarily a backdoor unless it also provides an unauthorized access/bypass mechanism.

Risks include theft of customer financial data, fraud targeting, regulatory/contractual exposure and loss of trust. The program's ordinary database/email privileges may make the action appear legitimate to coarse controls.

Use independent code review, separation of development/deployment authority, least-privilege database/mail access, controlled builds and monitoring for unusual outbound data. Maintain protected audit records so a privileged developer cannot erase the evidence easily.

A114 · 2018–19 · Q2(a)

Polymorphic versus metamorphic viruses with examples · 7 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Encrypted, polymorphic and metamorphic viruses.

> **Question summary:** Distinguish polymorphic and metamorphic viruses and give two examples of each.

Exam tip. "Changes its signature" describes both. State whether the body itself is rewritten.

A **polymorphic** virus varies its encrypted representation and often its decryptor while reconstructing substantially the same body at runtime. **Metamorphic** malware rewrites the body itself using equivalent instruction sequences, register changes, code permutation or junk insertion, so there need not be a constant decrypted body to recognize.

Historical examples commonly analyzed as polymorphic include **1260** and **Marburg**; metamorphic examples include **W32/Simile (Etap)** and **Win95/Regswap**. Families can combine techniques, so the mechanism is a more reliable distinction than a rigid label.

Polymorphic detection can emulate the decryptor and inspect the recovered body; metamorphic detection may require normalization, semantic/behavioral analysis and broader heuristics. Neither kind guarantees evasion against patching or behavior-based containment.

Marburg analysis, metamorphic analysis.

A115 · 2018–19 · Q2(b)

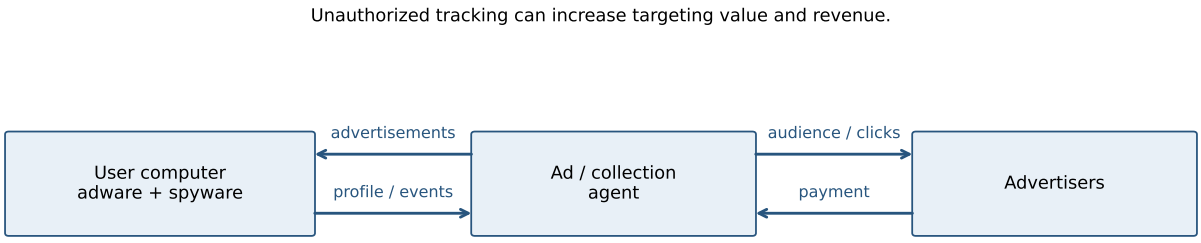
Why adware authors add spyware · 8 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Adware, spyware and malware economics.

Question summary: Explain why spyware benefits an adware author.

Exam tip. Label both the money flow and the information flow.



An installed adware component contacts an advertising agent, obtains advertisements and displays or injects them on the user's system. Advertisers pay the agent/operator for impressions, clicks or conversions.

Spyware adds collection of browsing interests, searches, application usage or other personal data. The operator can profile users, choose more profitable targeted ads and sell the data; abusive variants also steal credentials. More relevant ads can increase click/conversion revenue, explaining the incentive to combine the two components.

Keep the roles distinct: the **adware engine** displays ads, the **spyware engine** collects data, the **agent** distributes ads/receives reports, and **advertisers** fund the campaign. Not every ad-supported application secretly steals passwords; the security issue is unauthorized collection/actions and excessive privileges.

A116 · 2018–19 · Q2(c)

Function hooking and three concealment targets · 12 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Rootkit cloaking, hooking and DLL injection.

> **Question summary:** Define rootkit function hooking, its danger and three examples of avoiding detection.

Exam tip. Draw the intercepted query path and distinguish hooking from direct structure modification.

A rootkit provides concealment/persistence for other malicious activity. It can hide processes, files, services, registry entries, ports or user accounts. It is distinct from the payload it hides.

In a kernel hook, privileged malicious code redirects a kernel query's dispatch path or alters its result. For example:

```
process-list tool -> system query -> hooked enumeration routine
                                  -> remove malicious process record
                                  -> return a plausible filtered list
```

The process may keep executing while ordinary tools report it absent. File/directory and network enumeration can be filtered similarly. Installing a kernel hook normally needs elevated privileges or a separate privilege-escalation vulnerability. A user-mode API hook affects fewer observation paths; **DKOM** instead directly alters kernel bookkeeping structures and need not hook the query itself.

Detect with cross-view comparisons, integrity checks and trusted offline examination. A compromised kernel can lie to tools running under it, so ordinary successful antivirus output is not proof of a clean system. Recover from trusted media and rebuild when confidence cannot be restored.

Three concrete filtered observations are: (1) omit a malicious process from process enumeration; (2) hide its files in directory/file queries; (3) omit its listening port from network-connection queries. A service/registry query can be filtered too. These are concealment examples; they do not make all independent observation paths disappear.

A117 · 2018–19 · Q2(d)

Morris worm speed and consequences · 8 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Worm propagation, epidemic models and case studies.

> **Question summary:** Why did the Morris worm spread rapidly and what were the consequences?

Exam tip. Include reinfection/resource exhaustion, not just "it exploited a buffer overflow."

The 1988 Morris worm used several propagation routes, including a `fingerd` buffer overflow, a Sendmail debugging facility, trusted remote execution and password guessing. Multiple routes expanded the susceptible population, and automatic replication avoided the need for each new user to launch an attachment.

Its reinfection behavior caused repeated copies to run on already affected hosts. The resulting process/load growth exhausted resources and disrupted services across many connected systems, even without a conventional file-destruction payload. Widespread connectivity, trust relationships and weak/unpatched services amplified the effect.

The malware slides distinguish propagation vectors from payload: rapid scanning/replication is the spread mechanism; overload and loss of service are the damage.

A118 · 2018–19 · Q3(a)

Easter egg versus logic bomb · 6 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Backdoors, logic bombs and insider abuse.

> **Question summary:** Give examples of an Easter egg and a logic bomb.

Exam tip. Name the trigger and action for the logic bomb.

An **Easter egg** is a hidden, usually benign feature activated by an unusual input sequence—for example, a secret credits screen in a game. Hidden functionality still needs review because it may violate policy or introduce defects.

A **logic bomb** executes malicious behavior when a condition is met—for example, deleting project files on a chosen date or when a programmer's name disappears from payroll. It may lie dormant until its trigger occurs.

Both can be deliberately hidden; the essential distinction is the intended harmful action and trigger, not whether ordinary users know the feature exists. A backdoor specifically creates an unauthorized access/bypass path and need not be identical to either example.

A121 · 2018–19 · Q3(d)

Bot creation, control and uses · 10 marks

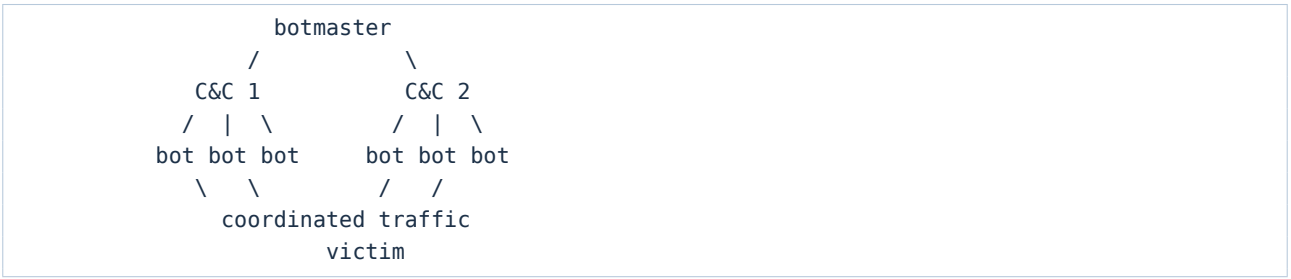
Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Botnet creation, C&C and malicious agents.

> **Question summary:** Explain botnet creation/control and possible activities.

Exam tip. Separate infection/agent installation, C&C command delivery and the eventual flood payload.

A botnet is a collection of compromised hosts running remotely controlled agents. A botmaster distributes instructions via one or more C&C endpoints or a peer-to-peer overlay. Agents report status and execute commands, often without their owners' knowledge.



For DDoS, many bots simultaneously send requests/packets that exhaust the victim's link, state, CPU or application capacity. Distribution increases aggregate rate and makes simple source-IP blocking costly because legitimate users can share networks with bots. Spoofing is optional: many botnet floods use the bots' real addresses.

Defend with endpoint prevention/cleanup, segmentation, C&C disruption, rate/admission controls and upstream traffic mitigation. A perimeter filter alone cannot restore an already saturated incoming link.

Other bot activities include spam, click fraud, credential/key logging and distributing additional malware. A P2P overlay can replace a single central C&C hierarchy and make disruption more difficult.

Network Security

Primary deck: Ch05-NetworkSecurity.pdf

A004 · 2023–24 · Q2(a)

Rogue DHCP gateway and MITM with the correct gateway · 10 marks

Original: past-paper PDF p. 3. Sitting date: 06 September 2025. Printed Section A.

Topics: DHCP starvation, spoofing and defenses; ARP spoofing, MITM and detection.

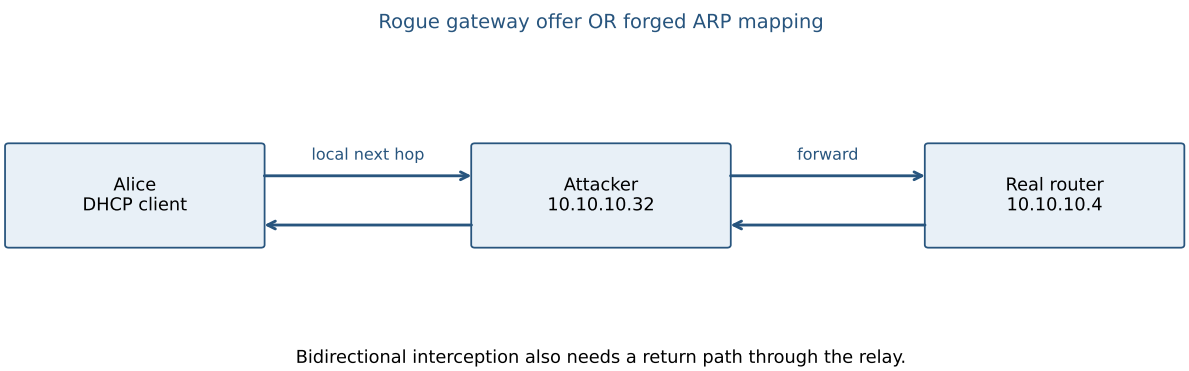
> **Question summary:** Attacker IP is 10.10.10.32; router/DHCP server is 10.10.10.4. Give a forged client IP/default router for Alice. If the correct router is supplied, explain another way to become MITM.

Exam tip. IP addresses choose routing destinations; Ethernet MAC addresses choose local next hops. Explain both when the real gateway IP is retained.

Offer an unused host address on the same subnet, for example **10.10.10.33**, and default router **10.10.10.32** (the attacker). The mask must make both reachable—for example /24 if consistent with the real network. Do not assign Alice an address already used by the router or attacker.

Alice then sends off-subnet packets to the attacker. To maintain a working connection, the attacker forwards them to 10.10.10.4. Routing/NAT or reverse-path manipulation is needed for replies to pass through the attacker as well; merely receiving outbound packets does not automatically provide a bidirectional MITM.

If the forged offer gives the correct router, use **ARP poisoning**: tell Alice that 10.10.10.4 has the attacker's MAC, and tell the router that Alice's IP has the attacker's MAC. Forward between the two. A malicious DNS-server option can instead redirect selected hostname traffic, but does not inherently intercept every connection and does not defeat properly validated TLS.



Defend with DHCP snooping (server replies only from trusted ports), request rate limits, binding-based dynamic ARP inspection and authenticated application protocols.

A006 · 2023–24 · Q2(c)

Off-path TCP injection and establishing a replacement connection · 15 marks

Original: past-paper PDF p. 3. Sitting date: 06 September 2025. Printed Section A.

Topics: TCP hijacking, spoofed segments and optimistic ACKs.

> **Question summary:** Using the printed TCP diagram (continued on p. 4), evaluate malicious actions by an off-path attacker and impersonating the client after it resets the connection.

Coverage: related application/extension. The network deck covers TCP state/hijacking; the modern blind-reset and ISN details extend it.

Exam tip. Separate "can spoof a packet," "TCP accepts it," and "application accepts the identity." They are three different claims.

An off-path attacker normally cannot see X, Y, the ephemeral client port, or current windows. The diagram's symbolic sequence numbers are explanatory; they are **not an address leak to the attacker**. Spoofing the client's IP is insufficient to inject accepted bytes reliably.

If the tuple and current numbers become known or predictable, a client-to-server injected segment after DATA C needs the next client sequence number

$$q = X + 1 + L_A + L_C,$$

and an ACK consistent with received server bytes, initially $Y + 1 + L_B$. Acceptance also depends on the receive window, TCP state and later traffic. A guessed in-window RST may disrupt legacy implementations; exact-sequence validation/challenge ACKs make blind resets harder. Flooding remains possible without successful hijacking.

For a **new** spoofed connection, the attacker can send a SYN under the client's source IP, but the server's SYN-ACK goes to the real client. To complete the handshake, the attacker must learn or predict the new server ISN and acknowledge it, and deal with the real client's resets. A prior connection's Y does not generally predict the next ISN. If the server authenticates clients at the application/TLS layer, a successful TCP handshake is still not authenticated impersonation.

Thus no reliable replacement-connection attack follows from the stated off-path capability alone. State additional assumptions if describing the classic predictable-ISN attack.

A010 · 2023–24 · Q4(a)

UDP chargen/echo feedback versus TCP handshake · 10 marks

Original: past-paper PDF p. 5. Sitting date: 06 September 2025. Printed Section A.

Topics: DoS/DDoS, reflection and resource exhaustion; TCP hijacking, spoofed segments and optimistic ACKs.

> **Question summary:** JULY on UDP port 36 returns 36–360 random characters; PROTIDHONI on port 5 echoes its input. Give one empty UDP packet creating a feedback loop between X and Y, then assess a single spoofed TCP SYN when both services require a handshake.

Exam tip. Trace every reply's source and destination ports. UDP has no handshake gate; TCP application services do.

Send the empty UDP datagram with:

Field	Value
Source IP / port	Y / 5
Destination IP / port	X / 36

X sends random characters back to Y:5. Y echoes them to X:36, provoking another random response, and so on. Both services consume bandwidth without further attacker packets, assuming they answer these requests indefinitely and packets continue to arrive. Reflection and application feedback create the problem; source spoofing hides the initiating sender.

With TCP, a single spoofed SYN does **not** create the same application-data loop. The recipient sends SYN-ACK to the spoofed peer, which has no matching connection and typically sends RST (or drops it). No three-way handshake/application request has completed. Since the attacker is off-path, it cannot generally learn the server sequence number needed to complete a forged handshake. There are therefore no magic seq/ACK values satisfying the requested one-SYN attack under the stated assumptions.

Prevent feedback by disabling unused services, refusing inappropriate service-to-service responses, limiting rates and applying source-address validation.

A022 · 2022–23 · Q1(b)

SYN backlog exhaustion and cookie validation · 12 marks

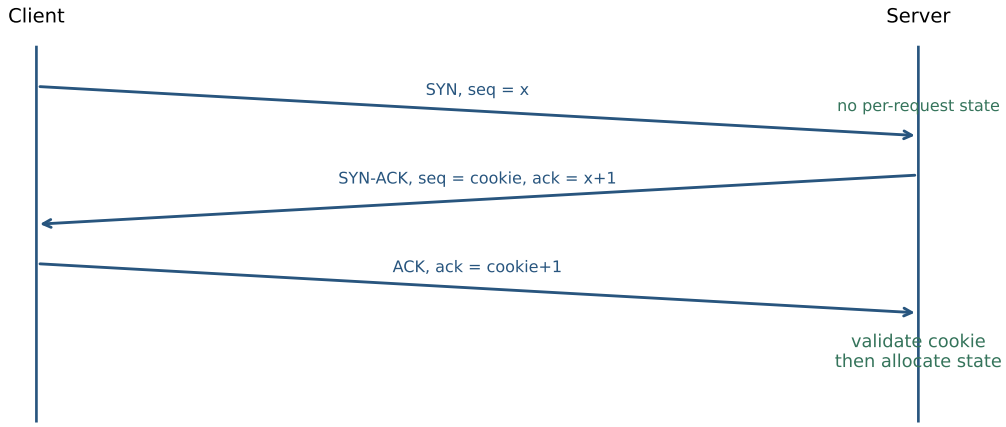
Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: SYN flooding and stateless SYN cookies.

Question summary: Explain SYN flooding with a figure and evaluate SYN cookies.

Exam tip. Explain exactly when state is allocated and why returning the cookie is evidence of receiving the SYN-ACK.

A normal TCP listener reserves state for a half-open connection after SYN and awaits the final ACK. Many incomplete handshakes can fill its backlog and deny legitimate clients.



Instead of storing each request, the server chooses an initial sequence number encoding a short time value, supported parameters and a secret-dependent authenticator of the flow tuple. It sends SYN-ACK with this cookie and ACK equal to client ISN plus one. When the client returns ACK, the server checks **ACK−1**, recomputes the authenticator for recent time slots, and allocates connection state only after validation.

Conceptually:

$$c = \mathrm{Encode}\,(t, \mathrm{options}, \mathrm{TruncMAC}_K(\mathrm{srcIP}, \mathrm{srcPort}, \mathrm{dstIP}, \mathrm{dstPort}, t, \mathrm{options}))\,.$$

The real cookie fits within the 32-bit TCP ISN, so authentication bits and options are limited. The secret is not sent to clients. A blind spoofing sender generally cannot see and return a correct cookie. An on-path adversary can, and attackers with genuine addresses can still complete handshakes and consume resources.

Cookies protect half-open **state**, not an already saturated link, all CPU costs, or application floods. Historical slide statements about OS support/options are not universal claims about current implementations.

Reference: RFC 4987.

A023 · 2022–23 · Q1(c)

DHCP starvation message flow and rogue-server relationship · 13 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: DHCP starvation, spoofing and defenses.

> **Question summary:** Explain DHCP starvation with a message diagram and analyze its relation to DHCP spoofing.

Exam tip. Label starvation as DoS and rogue configuration as a separate trust failure.

Normal allocation follows **DISCOVER → OFFER → REQUEST → ACK**. A client broadcasts discovery; a server offers a lease; the client selects it; the server acknowledges.

In starvation, an attacker presents many invented client identities and completes enough allocation exchanges to occupy the available lease pool. Legitimate new clients then receive no usable lease. Many DISCOVERs alone may consume pending offers temporarily, but a complete explanation includes lease commitment/retention behavior.

```
invented client i  -> server: DISCOVER
server            -> client: OFFER address i
invented client i  -> server: REQUEST address i
server            -> client: ACK lease i
repeat until the allocatable pool is exhausted
```

A rogue DHCP server can then offer an attacker-controlled gateway or DNS resolver to clients the legitimate server cannot serve. This **spoofing/MITM** phase is distinct from starvation's availability failure; spoofing can also succeed by winning a race without first exhausting the pool.

Use DHCP snooping with trusted server ports, rate limits on untrusted client ports, identity/lease limits and monitoring for abnormal allocation churn. Port-security limits alone may not cover all DHCP client-identifier variations.

A027 · 2022–23 · Q3(a)

CAM exhaustion and switch behavior · 12 marks

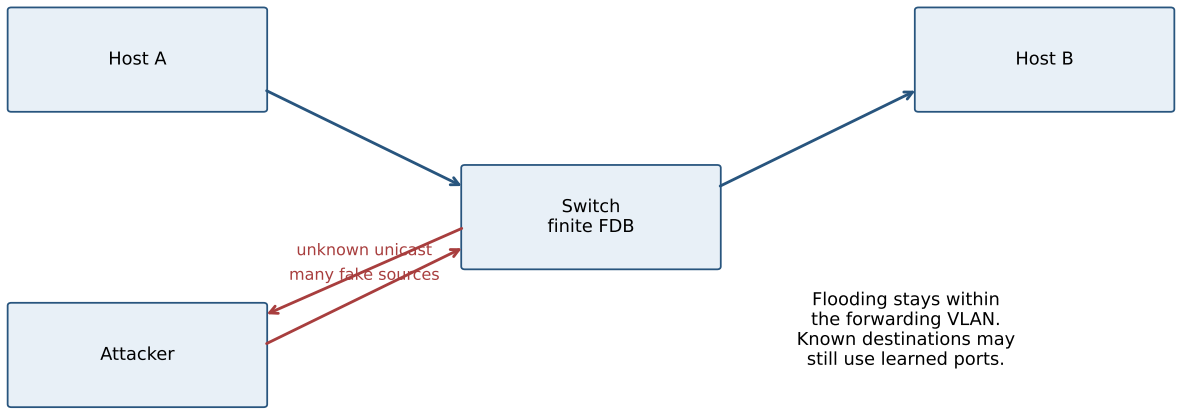
Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Switch MAC-table flooding and port security.

> **Question summary:** Explain MAC flooding with topology, effects on a switch and attacker benefits.

Exam tip. Explain source learning and destination lookup separately; exhausting an ARP cache is a different attack.

A switch learns **source MAC → ingress port** bindings in its finite CAM/FDB table. An attacker sends frames with many fabricated source MACs, exhausting or churning that table.



When a destination MAC is unknown, ordinary switching may flood that frame to other forwarding ports **within its VLAN**. The attacker can then see traffic that would otherwise have been unicast elsewhere, and excessive flooding may cause congestion. This does not universally turn the device into a hub: existing known destinations may still be forwarded correctly, hardware behavior varies and VLAN boundaries remain relevant.

Use port-security MAC limits, appropriate violation actions, source controls, segmentation and unknown-unicast limits. Monitor abnormal source churn/FDB pressure. End-to-end encryption protects contents even if frames are copied. DAI addresses forged ARP bindings and is not a complete substitute for CAM-source limits.

A060 · 2021–22 · Q6(a)

Detect optimistic ACKs by withholding a segment · 12 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: TCP hijacking, spoofed segments and optimistic ACKs.

> **Question summary:** Explain how randomly dropping data segments at the sender helps detect optimistic TCP ACK attacks.

Coverage: related application/extension. The network deck explains TCP ACK state; optimistic-ACK probing is beyond its explicit treatment.

Exam tip. "Randomly drop packets" is incomplete: explain the impossible acknowledgment that supplies evidence.

An optimistic receiver ACKs bytes it has not received, tricking a sender into increasing its congestion window and sending faster than the path can carry. An attacker can abuse this to amplify offered traffic.

The proposed sender defense secretly **withholds** a chosen segment before it ever enters the network and tracks the resulting byte gap. Suppose bytes 1000–1999 are withheld while later segments arrive. A correct cumulative ACK cannot advance beyond 1000 until those missing bytes have actually been transmitted and received (ACK numbers denote the next expected byte).

A receiver that acknowledges 3000 while the gap still exists claims receipt of bytes the sender never sent. The sender can flag/terminate or restrict that flow. Random selection makes it harder for a blind ACK generator to predict the probes. Merely observing network loss is weaker because retransmission/reordering can explain later ACKs; track exactly what was intentionally withheld and when retransmission becomes possible.

This is the historical proposed modification in the question, not an assumption that all TCP stacks implement it. It must account for legitimate cumulative ACK/SACK behavior and imposes some performance cost.

Primary study, CERT vulnerability note.

A063 · 2021–22 · Q7(a)

SYN-cookie defense · 12 marks

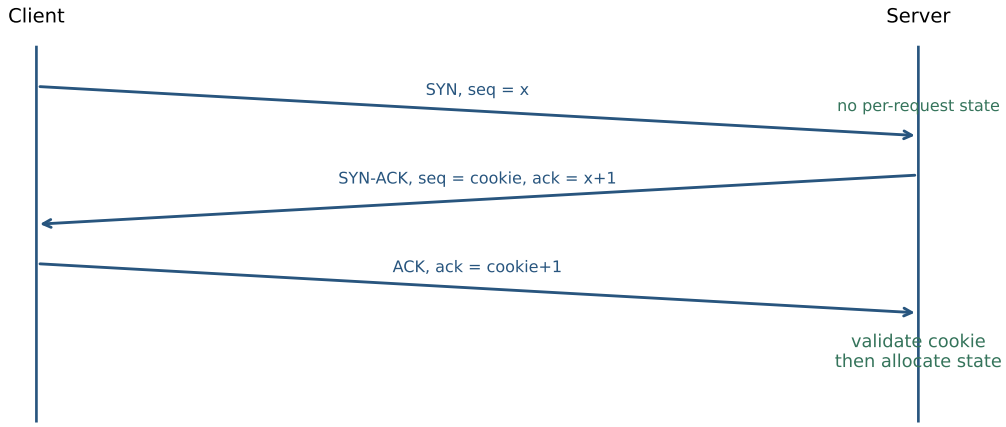
Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: SYN flooding and stateless SYN cookies.

> **Question summary:** Explain how SYN cookies defend against SYN flooding.

Exam tip. Explain exactly when state is allocated and why returning the cookie is evidence of receiving the SYN-ACK.

A normal TCP listener reserves state for a half-open connection after SYN and awaits the final ACK. Many incomplete handshakes can fill its backlog and deny legitimate clients.



Instead of storing each request, the server chooses an initial sequence number encoding a short time value, supported parameters and a secret-dependent authenticator of the flow tuple. It sends SYN-ACK with this cookie and ACK equal to client ISN plus one. When the client returns ACK, the server checks **ACK−1**, recomputes the authenticator for recent time slots, and allocates connection state only after validation.

Conceptually:

$$c = \mathrm{Encode}\,(t, \mathrm{options}, \mathrm{TruncMAC}_K(\mathrm{srcIP}, \mathrm{srcPort}, \mathrm{dstIP}, \mathrm{dstPort}, t, \mathrm{options}))\,.$$

The real cookie fits within the 32-bit TCP ISN, so authentication bits and options are limited. The secret is not sent to clients. A blind spoofing sender generally cannot see and return a correct cookie. An on-path adversary can, and attackers with genuine addresses can still complete handshakes and consume resources.

Cookies protect half-open **state**, not an already saturated link, all CPU costs, or application floods. Historical slide statements about OS support/options are not universal claims about current implementations.

Reference: RFC 4987.

A078 · 2020–21 · Q4(a)

ARP prevention, monitoring and locating the attacking port · 12 marks

Original: past-paper PDF p. 19. Sitting date: 25 September 2022. Printed Section A.

Topics: ARP spoofing, MITM and detection.

> **Question summary:** Give ARP-spoofing defenses and ways to identify an attacking host on the internal network.

Exam tip. Separate detecting a bad binding from attributing it to a physical port/device.

ARP maps an IPv4 next-hop address to an Ethernet MAC without strong native authentication. Forged replies can tell a victim that the gateway's IP belongs to the attacker and tell the gateway the inverse lie, creating a forwarding MITM.

Prevent with DHCP-snooping-derived **dynamic ARP inspection**, carefully maintained static bindings for small critical environments, appropriate switch access controls and authenticated end-to-end protocols. Encryption protects payloads but does not remove false local routing or DoS.

Monitor ARP changes, duplicate claims for one IP, unusual gratuitous replies and binding conflicts using packet capture/ARPwatch-style tools. Compare with legitimate DHCP leases, inventory and known gateway MACs. Then correlate the suspicious MAC with the switch forwarding table, ingress-port counters and physical/802.1X identity to find its access port; isolate according to the response plan.

A single MAC serving several IPs is an indicator rather than proof: routers, virtualization and failover can do that legitimately. MACs can also be spoofed; do not equate a packet's source string with a trustworthy person identity.

A099 · 2019–20 · Q4(b)

Making a switch flood unknown destinations · 12 marks

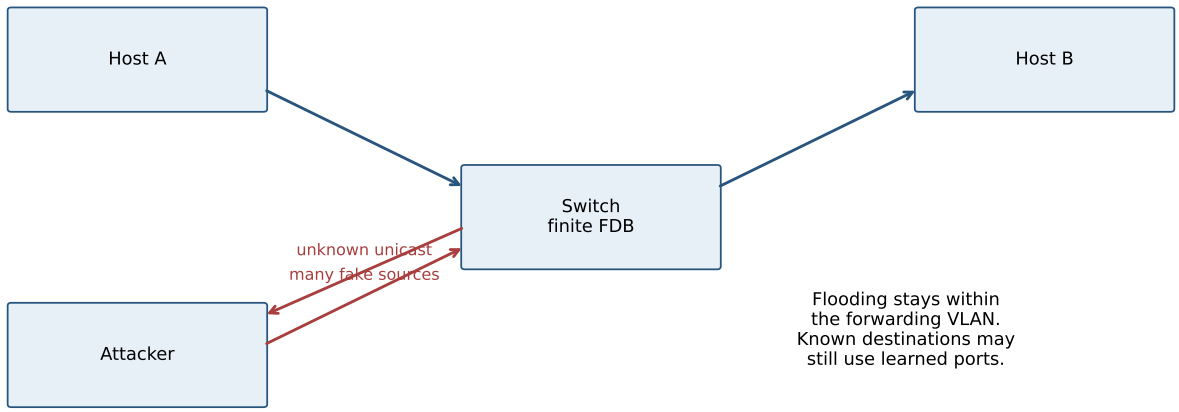
Original: past-paper PDF p. 23. Sitting date: 07 September 2021. Printed Section A.

Topics: Switch MAC-table flooding and port security.

> **Question summary:** Why make a switch behave like a hub, what attack does this and how can it be prevented?

Exam tip. Explain source learning and destination lookup separately; exhausting an ARP cache is a different attack.

A switch learns **source MAC → ingress port** bindings in its finite CAM/FDB table. An attacker sends frames with many fabricated source MACs, exhausting or churning that table.



When a destination MAC is unknown, ordinary switching may flood that frame to other forwarding ports **within its VLAN**. The attacker can then see traffic that would otherwise have been unicast elsewhere, and excessive flooding may cause congestion. This does not universally turn the device into a hub: existing known destinations may still be forwarded correctly, hardware behavior varies and VLAN boundaries remain relevant.

Use port-security MAC limits, appropriate violation actions, source controls, segmentation and unknown-unicast limits. Monitor abnormal source churn/FDB pressure. End-to-end encryption protects contents even if frames are copied. DAI addresses forged ARP bindings and is not a complete substitute for CAM-source limits.

A100 · 2019–20 · Q4(c)

Withheld-segment detection of optimistic ACKs · 11 marks

Original: past-paper PDF p. 23. Sitting date: 07 September 2021. Printed Section A.

Topics: TCP hijacking, spoofed segments and optimistic ACKs.

> **Question summary:** Explain with examples how sender-side random segment dropping detects optimistic ACKs.

Coverage: related application/extension. Optimistic-ACK details extend the TCP-state slides.

Exam tip. "Randomly drop packets" is incomplete: explain the impossible acknowledgment that supplies evidence.

An optimistic receiver ACKs bytes it has not received, tricking a sender into increasing its congestion window and sending faster than the path can carry. An attacker can abuse this to amplify offered traffic.

The proposed sender defense secretly **withholds** a chosen segment before it ever enters the network and tracks the resulting byte gap. Suppose bytes 1000–1999 are withheld while later segments arrive. A correct cumulative ACK cannot advance beyond 1000 until those missing bytes have actually been transmitted and received (ACK numbers denote the next expected byte).

A receiver that acknowledges 3000 while the gap still exists claims receipt of bytes the sender never sent. The sender can flag/terminate or restrict that flow. Random selection makes it harder for a blind ACK generator to predict the probes. Merely observing network loss is weaker because retransmission/reordering can explain later ACKs; track exactly what was intentionally withheld and when retransmission becomes possible.

This is the historical proposed modification in the question, not an assumption that all TCP stacks implement it. It must account for legitimate cumulative ACK/SACK behavior and imposes some performance cost.

Primary study, CERT vulnerability note.

A119 · 2018–19 · Q3(b)

Complete TCP hijacking · 9 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: TCP hijacking, spoofed segments and optimistic ACKs; ARP spoofing, MITM and detection.

Question summary: Explain with a diagram how to perform TCP session hijacking.

Exam tip. Separate observing numbers, injecting acceptable bytes and silencing conflicting legitimate traffic.

An attacker first needs the connection tuple and sequence/window state. A practical on-LAN route in the lecture is ARP poisoning, which makes the attacker a forwarding intermediary:

```
client <----> attacker relay <----> server
              observe seq/ACK
              suppress/replace selected packets
```

Track bytes in both directions. To impersonate the client, a forged segment must use the client's source IP/port, server destination and an acceptable next sequence number/ACK. Its TCP checksum must match. If the real client continues sending conflicting stream data, the session can desynchronize or terminate; full takeover requires controlling/suppressing that competing traffic, not merely inserting one arbitrary packet.

Correct authenticated application encryption prevents accepted malicious **application content** without the session keys, although an on-path attacker may still disrupt availability. Random ISNs and blind-injection defenses help against off-path guessing but do not hide state from an on-path observer.

A120 · 2018–19 · Q3(c)

Optimistic TCP ACK mechanism and mitigation · 10 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: TCP hijacking, spoofed segments and optimistic ACKs.

> **Question summary:** Define optimistic TCP ACK attack and give mitigations.

Coverage: related application/extension. The deck covers TCP state but not optimistic-ACK algorithms.

Exam tip. "Randomly drop packets" is incomplete: explain the impossible acknowledgment that supplies evidence.

An optimistic receiver ACKs bytes it has not received, tricking a sender into increasing its congestion window and sending faster than the path can carry. An attacker can abuse this to amplify offered traffic.

The proposed sender defense secretly **withholds** a chosen segment before it ever enters the network and tracks the resulting byte gap. Suppose bytes 1000–1999 are withheld while later segments arrive. A correct cumulative ACK cannot advance beyond 1000 until those missing bytes have actually been transmitted and received (ACK numbers denote the next expected byte).

A receiver that acknowledges 3000 while the gap still exists claims receipt of bytes the sender never sent. The sender can flag/terminate or restrict that flow. Random selection makes it harder for a blind ACK generator to predict the probes. Merely observing network loss is weaker because retransmission/reordering can explain later ACKs; track exactly what was intentionally withheld and when retransmission becomes possible.

This is the historical proposed modification in the question, not an assumption that all TCP stacks implement it. It must account for legitimate cumulative ACK/SACK behavior and imposes some performance cost.

Primary study, CERT vulnerability note.

Complementary mitigations include sensible pacing/traffic shaping and per-client resource limits; they reduce amplification but do not themselves prove receipt of every byte. The withheld-segment test specifically targets false receipt claims.

DNS Security

Primary deck: Ch06-1-NetworkDNS.pdf

A009 · 2023–24 · Q3(c)

Kaminsky retries, bailiwick, DNS-over-TLS and DNSSEC · 20 marks

Original: past-paper PDF p. 4. Sitting date: 06 September 2025. Printed Section A.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

> **Question summary:** An internal attacker loads many fakeN.buet.ac.bd image names to poison the shared resolver (no source-port randomization). Explain why many images help, whether the same page can poison www.google.com, and the effects of DNS-over-TLS and DNSSEC.

Exam tip. Identify which zone the forged record concerns and which network hop each defense actually protects.

Each fresh random subdomain forces a cache miss and creates another response race. The attacker attempts to match the outstanding query's name/type/class, transaction ID and socket tuple before the authoritative response arrives. A forged in-bailiwick delegation/NS response can try to redirect subsequent resolution for the parent zone. Repeating fresh names avoids waiting for one cached answer's TTL to expire.

For q independent races with per-race win probability p:

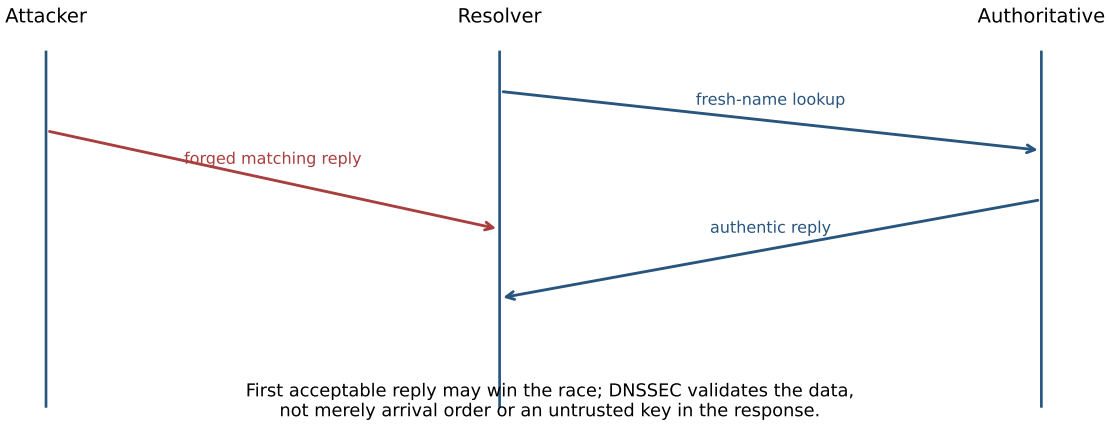
$$P(\text{at least one win}) = 1 - (1 - p)^q.$$

If r distinct ID guesses fit in a race and the only unknown is a uniform 16-bit ID, $p \approx \min(r, 2^{16})/2^{16}$. Sending all IDs helps only if they arrive in time. Do not apply an unrestricted $qr/2^{16}$ formula when names, sockets or implementation query coalescing prevent cross-matching.

The listed `fakeN.buet.ac.bd` queries do **not** authorize replacement of `google.com` data. A resolver enforcing bailiwick/trust rules discards unrelated additional records. Targeting another domain requires corresponding outstanding queries and an applicable poisoning opportunity; "all DNS is cached together" is insufficient.

DNS over TLS depends on the hop. Authenticating/encrypting the client-to-recursive-resolver connection prevents tampering on that link, but does not secure a separate UDP recursive-to-authoritative race. Authenticated protection of the attacked hop would stop network forgery there.

DNSSEC validates signed RRsets through DNSKEY/DS and a configured trust anchor. Forged answer/delegation data fails signature/chain validation for a securely signed zone. An untrusted key bundled in the answer is not a trust anchor. DNSSEC does not conceal queries or guarantee availability.



Reference: RFC 5452, DNSSEC introduction: RFC 4033.

A030 · 2022–23 · Q4(a)

DNS cache poisoning and three defenses · 12 marks

Original: past-paper PDF p. 12. Sitting date: 18 April 2024. Printed Section A.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

> **Question summary:** Define DNS cache poisoning, explain exploitation and provide three defenses.

Exam tip. Give a defense against the exact forgery path; “use a firewall” alone does not authenticate DNS data.

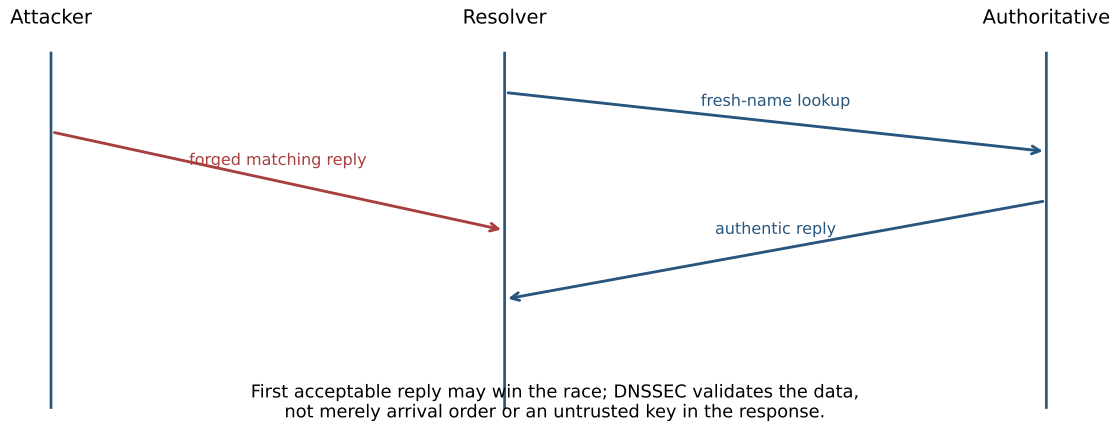
A resolver stores a forged name/delegation mapping and later serves it as cached truth. Clients may be redirected to an attacker’s server or denied service. An off-path attacker commonly forces an uncached lookup and races a guessed response against the authoritative reply.

The forged response must match the outstanding query’s transaction ID, name/type/class, addresses and ports and satisfy record-trust/bailiwick rules. A 16-bit predictable identifier or fixed source port makes guessing easier. A cached malicious delegation can affect many subsequent names.

Three complementary defenses are:

1. **Unpredictable transaction IDs and per-query source ports:** enlarge the guessing space; keep entropy through NAT/firewalls.
2. **Strict response and bailiwick validation:** reject mismatched/unsolicited responses and unrelated additional records; restrict recursion to intended clients and patch resolver flaws.
3. **DNSSEC validation:** verify signed RRsets through authenticated delegation to a trust anchor. Reject bogus data for securely signed zones.

Restricting recursion alone does not protect against an internal attacker or a browser-triggered lookup. DNSSEC provides authenticity/integrity, not encryption or total DoS prevention.



Reference: RFC 5452.

A061 · 2021–22 · Q6(b)

Three DNS poisoning countermeasures · 12 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

> **Question summary:** Design three mechanisms to prevent or mitigate DNS cache poisoning.

Exam tip. Give a defense against the exact forgery path; “use a firewall” alone does not authenticate DNS data.

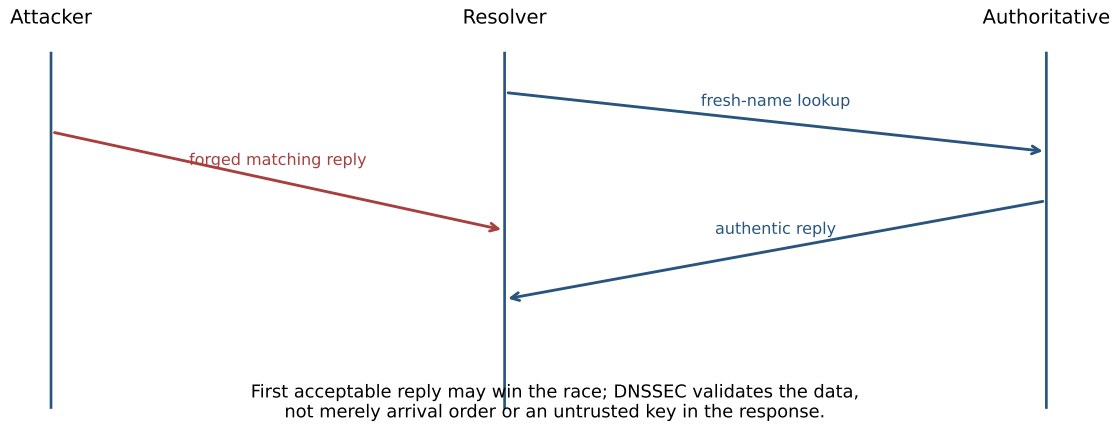
A resolver stores a forged name/delegation mapping and later serves it as cached truth. Clients may be redirected to an attacker’s server or denied service. An off-path attacker commonly forces an uncached lookup and races a guessed response against the authoritative reply.

The forged response must match the outstanding query’s transaction ID, name/type/class, addresses and ports and satisfy record-trust/bailiwick rules. A 16-bit predictable identifier or fixed source port makes guessing easier. A cached malicious delegation can affect many subsequent names.

Three complementary defenses are:

1. **Unpredictable transaction IDs and per-query source ports:** enlarge the guessing space; keep entropy through NAT/firewalls.
2. **Strict response and bailiwick validation:** reject mismatched/unsolicited responses and unrelated additional records; restrict recursion to intended clients and patch resolver flaws.
3. **DNSSEC validation:** verify signed RRsets through authenticated delegation to a trust anchor. Reject bogus data for securely signed zones.

Restricting recursion alone does not protect against an internal attacker or a browser-triggered lookup. DNSSEC provides authenticity/integrity, not encryption or total DoS prevention.



Reference: RFC 5452.

A098 · 2019–20 · Q4(a)

DNS redirection as a MITM precursor · 12 marks

Original: past-paper PDF p. 23. Sitting date: 07 September 2021. Printed Section A.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

Question summary: Explain with a diagram how DNS spoofing can launch MITM.

Exam tip. Explain the additional trust failure needed to turn DNS poisoning into interception of validated HTTPS.

```
client asks resolver: bank.example address?
forged/corrupted answer: attacker IP
client -> attacker proxy -> real bank
client <- attacker proxy <- real bank
```

The attacker replaces a hostname's expected address, receives the client's connection and proxies to the real service while observing/modifying unprotected traffic. A local forged answer or poisoned recursive cache can supply the wrong mapping. To be MITM, the attacker must forward both sides rather than merely show an unrelated fake page.

Correct HTTPS hostname/certificate validation prevents a DNS redirect alone from authenticating the attacker as `bank.example`: it lacks the trusted corresponding server key. It may still cause failure/DoS, exploit a user overriding warnings or succeed with a separately compromised certificate/key. DNSSEC validation addresses signed DNS authenticity; authenticated TLS independently binds the server identity.

A122 · 2018–19 · Q4(a)

Random-subdomain cache-poisoning advantage · 10 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

> **Question summary:** Why use subdomain DNS cache poisoning, and why can it succeed more readily than simple poisoning?

Exam tip. Explain the two advantages separately: repeated cache misses and a valuable cached delegation target.

A simple repeated query for one name may hit a cached answer, leaving no new authoritative response race until TTL expiry. Random fresh subdomains such as `nonce1.example`, `nonce2.example` repeatedly cause cache misses.

For each outstanding lookup, the attacker races forged replies carrying an applicable delegation/NS change for the target zone. A successful accepted delegation can redirect later lookups beyond that one fresh name. New names provide many attempts without waiting for the original host record to expire.

Success still requires matching the actual query identifiers/socket and satisfying resolver trust/bailiwick rules. This is not permission to insert arbitrary records for unrelated domains, and a correct DNSSEC-validating resolver rejects forged data for a securely signed zone. Unpredictable IDs/source ports, strict validation and resolver updates reduce the race opportunity.

A123 · 2018–19 · Q4(b)

DNS redirection in a pharming-versus-phishing comparison · 8 printed marks (mixed question)

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: DNS poisoning, Kaminsky races, pharming and DNSSEC.

> **Question summary:** Explain the DNS-redirection (pharming) side of the printed comparison, with an example.

Sec A scope: Covers the DNS/pharming component of the printed comparison. The phishing topic is assigned to Sec B. The printed 8 marks are for the complete comparison, not this component alone.

Coverage: related application/extension. Only the Sec A DNS/pharming component is developed here; the standalone phishing topic remains in Sec B.

Exam tip. Locate the altered mapping or resolver configuration. A fake-looking URL alone is not evidence of DNS poisoning.

Pharming redirects a legitimate hostname to an attacker-controlled destination by changing a trusted-looking name-resolution path. For example, a poisoned resolver cache maps `bank.example` to the attacker's address. A user can type the correct hostname and still reach that address; this distinguishes DNS redirection from a lure persuading the user to visit a deceptive hostname.

Trace the Sec A mechanism:

```
user requests correct hostname -> resolver returns poisoned mapping
                               -> client connects to attacker address
```

A poisoned hosts file, rogue resolver configuration or compromised router can produce a similar redirection. Use protected resolver configuration, validated DNS responses/DNSSEC where applicable and correctly authenticated endpoint connections. A DNS change alone does not supply the trusted server's private key or defeat proper certificate validation; denial of service may still occur.

Firewalls, Tunneling and IDS

Primary deck: Ch06-2-FW-Tunneling-IDS-REDUCED.pdf

A007 · 2023–24 · Q3(a)

Can NIDS/firewall plus HTTPS always stop the network attacks? · 10 printed marks (mixed question)

Original: past-paper PDF p. 4. Sitting date: 06 September 2025. Printed Section A.

Topics: Firewall state, evasion and defense limits; TCP hijacking, spoofed segments and optimistic ACKs; SYN flooding and stateless SYN cookies; DNS poisoning, Kaminsky races, pharming and DNSSEC; DoS/DDoS, reflection and resource exhaustion.

> **Question summary:** An HTTPS-only server farm has a NIDS and firewall processing incoming requests. With an on-path attacker, assess the Sec A cases: RST injection, SYN flooding, DNS spoofing and DDoS.

Sec A scope: Covers the four network-defense cases. The reflected-XSS case in the original five-part question belongs to Sec B and is excluded. The 10 printed marks cover all five cases; no per-case allocation is supplied.

Exam tip. Place each attack at its layer: link/IP, TCP, TLS or application. A protection at one layer need not authenticate the others.

No blanket guarantee follows from this setup. A NIDS may only alert; a firewall's effectiveness depends on its policy, visibility and resources.

Attack	Reason and relevant defense
RST injection	TLS authenticates records, not ordinary TCP headers. An on-path adversary can observe sequence state and attempt a valid reset. State checks/TCP-level authentication can help; HTTPS alone does not authenticate RSTs.
SYN flooding	SYNs reach the server before TLS. SYN cookies/proxies and upstream limits reduce backlog exhaustion, but not all link saturation.
DNS spoofing	HTTPS certificate validation can stop successful server impersonation when DNS redirects to an unauthenticated server. It does not prevent poisoning itself or resulting denial of service. DNSSEC validation addresses signed DNS data.
DDoS	A firewall may filter attack traffic but cannot recover bandwidth already exhausted upstream. Distributed mitigation, capacity and application limits are needed.

A011 · 2023–24 · Q4(b)

TCP segmentation, reassembly and limits of a guaranteed evasion · 10 marks

Original: past-paper PDF p. 5. Sitting date: 06 September 2025. Printed Section A.

Topics: Firewall state, evasion and defense limits.

> **Question summary:** A smart stateful firewall blocks bytes spelling admin and allows everything else. Mallory splits the word into five packets sent out of order. Will that evade the rule? If not, propose a guaranteed strategy.

Coverage: related application/extension. The deck distinguishes filter types; stream normalization and the impossibility of an unconditional guarantee are extensions.

Exam tip. State the inspection model first. "Stateful" does not automatically mean complete application parsing, and "encrypted" does not automatically mean permitted.

If the firewall scans individual packets only, splitting `admin` prevents any one packet from containing the complete pattern. That can evade this **packet-only implementation**, whether packets arrive in order or not.

A stateful **stream-inspecting** firewall can buffer TCP segments, order them by sequence number, normalize overlaps and scan the reassembled byte stream. It reconstructs `admin` just as the endpoint does. Reordering then provides no inherent evasion.

There is **no universally guaranteed bypass** of the specified smart firewall. Encryption or an encoding can hide literal bytes only if the receiving application accepts that representation and the firewall cannot decrypt/decode it. For example, a permitted authenticated tunnel can carry an encrypted `admin`, but this assumes tunnel support, keys and a policy allowing that traffic. An application-aware firewall might inspect after decryption or reject it.

Likewise, fragmentation/overlap tricks depend on a disagreement between firewall and endpoint parsers; correctly matched reassembly defeats them. For an ideal filter that sees exactly the delivered application stream and rejects this word, delivering that same visible word while evading the rule is contradictory.

A026 · 2022–23 · Q2(c)

Fingerprint/version/CVE assessment workflow · 11 marks

Original: past-paper PDF p. 11. Sitting date: 18 April 2024. Printed Section A.

Topics: Reconnaissance, fingerprinting and vulnerability assessment.

> **Question summary:** Formulate the steps for finding and exploiting a target vulnerability by scanning OS/application versions.

Exam tip. Separate discovery, vulnerability inference and demonstrated exploitation. Each has different evidence.

In the exam's authorized test environment:

1. Define the permitted hosts/services and obtain a baseline of expected availability.
2. Discover live hosts and open ports. A TCP connect scan completes a connection; a SYN scan interprets SYN-ACK/RST without completing an ordinary connection. Filtering/timeouts require careful interpretation.
3. Identify service banners/protocol behavior and estimate OS/version using tools such as Nmap. A banner is evidence, not proof: it can be hidden, misleading or refer to a backported patched build.
4. Match the exact product, version, configuration and platform against vendor advisories/CVE records. Check authentication requirements, exposed interfaces and patch status.
5. Validate the suspected issue safely in an isolated equivalent system, then demonstrate only the agreed impact on the test target. A version match alone does not establish successful code execution.
6. Preserve evidence, explain the root cause, patch or reconfigure, reduce exposure and retest the specific finding.

The slides mention scanners/exploit tools; a good answer connects each tool's observation to the next decision rather than treating a scan result as automatic access.

A031 · 2022–23 · Q4(b)

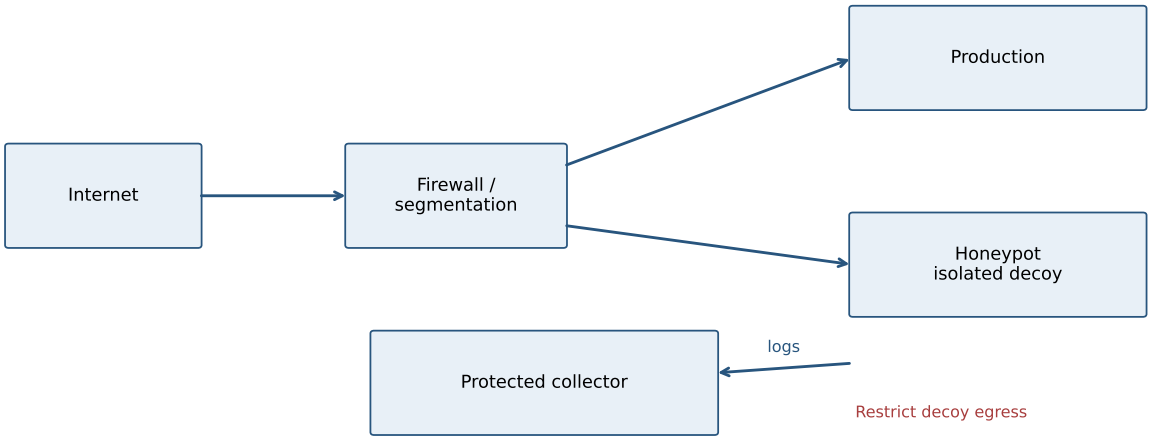
Honeypot placement, value and resource cost · 10 marks

Original: past-paper PDF p. 12. Sitting date: 18 April 2024. Printed Section A.

Topics: Honeypot topology, monitoring and overhead.

> **Question summary:** Explain with topology why an organization needs a honeypot and estimate its overhead.

Exam tip. Distinguish direct production latency from operational and shared-resource cost.



A honeypot is a deliberately attractive decoy with little or no legitimate production use. Access attempts provide comparatively high-signal evidence. Interaction can reveal exploit methods, malware, commands and C&C destinations, and may divert an intruder from valuable systems.

Place it in a controlled segment/DMZ, monitor it from a protected collector and restrict its outbound reach. Never give it real credentials or a path for unrestricted attacks on production or third parties. A low-interaction decoy emulates selected services; a high-interaction host provides richer evidence at greater containment/maintenance cost.

Overhead is not zero and has no fixed percentage in the question. Costs include a VM/host, storage, logs, monitoring, analyst time, patch/reset work and some network traffic. A passive or separately segmented decoy need not sit in the production request path, so ordinary requests do not inherently incur an extra hop. Attack traffic or poor placement can nevertheless affect shared resources.

A honeypot observes attacks that touch it; it cannot replace general IDS, patching and access control.

A032 · 2022–23 · Q4(c)

Stateful versus stateless filtering · 7 marks

Original: past-paper PDF p. 12. Sitting date: 18 April 2024. Printed Section A.

Topics: Firewall state, evasion and defense limits.

> **Question summary:** Compare stateful and stateless firewalls.

Exam tip. Give an example packet accepted by a simplistic stateless rule but rejected for lack of state.

Property	Stateless filter	Stateful filter
Decision input	Current packet's fields and rules	Packet plus tracked flow/ connection context
Return traffic	Must be allowed by explicit broad rules	Can allow replies to an established permitted flow
Unsolicited SYN-ACK	Could match a permissive source-port/flag rule	Can reject if no matching connection state exists
Resource use	Usually lower per-flow storage	State table consumes memory and can be exhausted
Visibility limit	Header rules do not establish application safety	Being part of a flow does not make its payload safe

Stateful filtering follows TCP transitions/tuples and may validate sequence windows; exact tracking varies. UDP "state" generally means inferred tuple/time-based associations, not a UDP handshake. Application parsing and TLS decryption are additional capabilities, not guaranteed by the term stateful.

A058 · 2021–22 · Q5(b)

AH versus ESP and transport/tunnel layouts · 12 marks

Original: past-paper PDF p. 16. Sitting date: 30 October 2023. Printed Section B.

Topics: VPN and IPsec AH/ESP encapsulation.

> **Question summary:** Compare IPsec AH and ESP with their encapsulations.

Exam tip. Mark the exact encrypted/authenticated region instead of saying simply "AH authenticates, ESP encrypts."

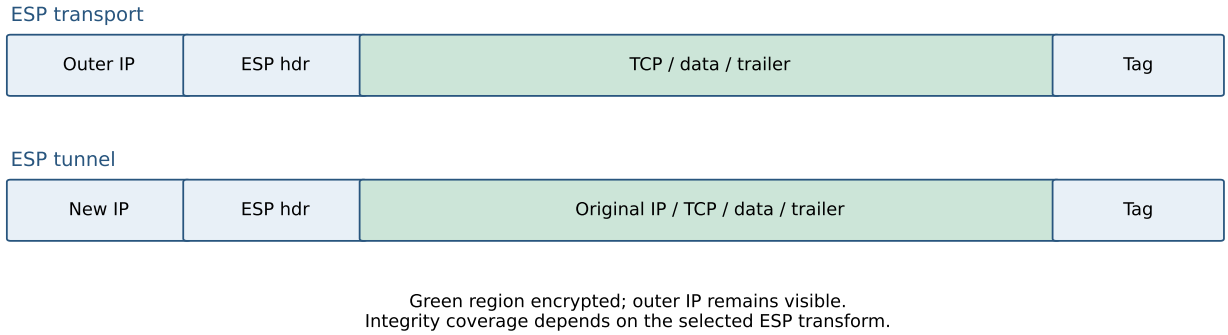
AH provides packet integrity/data-origin authentication and anti-replay when enabled, but no confidentiality. Its authentication covers payload and selected immutable/predictable IP-header fields; mutable fields cannot be protected unchanged across transit.

ESP protects the encapsulated payload with confidentiality and, when configured, integrity/authentication (normally combined in secure deployments). It does not authenticate the outer IP header in the same way as AH. Anti-replay uses sequence numbers and a receiver window, with authenticated integrity necessary to trust them.

```
AH transport: IP | AH | TCP/UDP | data
AH tunnel:    new IP | AH | original IP | TCP/UDP | data
ESP transport: IP | ESP hdr | {TCP/UDP | data | ESP trailer} | auth tag
ESP tunnel: new IP | ESP hdr | {original IP | TCP/UDP | data | trailer} | tag
```

Braces mark the ESP-encrypted part when encryption is selected. ESP header fields such as SPI/sequence number remain visible but are integrity-protected when authentication is used. Tunnel mode hides the inner addresses from ordinary transit observation while outer endpoints remain visible.

AH's protection of address fields makes ordinary NAT problematic. ESP can traverse NAT using UDP encapsulation; do not claim all IPsec is intrinsically incompatible with NAT.



References: RFC 4302, RFC 4303.

A062 · 2021–22 · Q6(c)

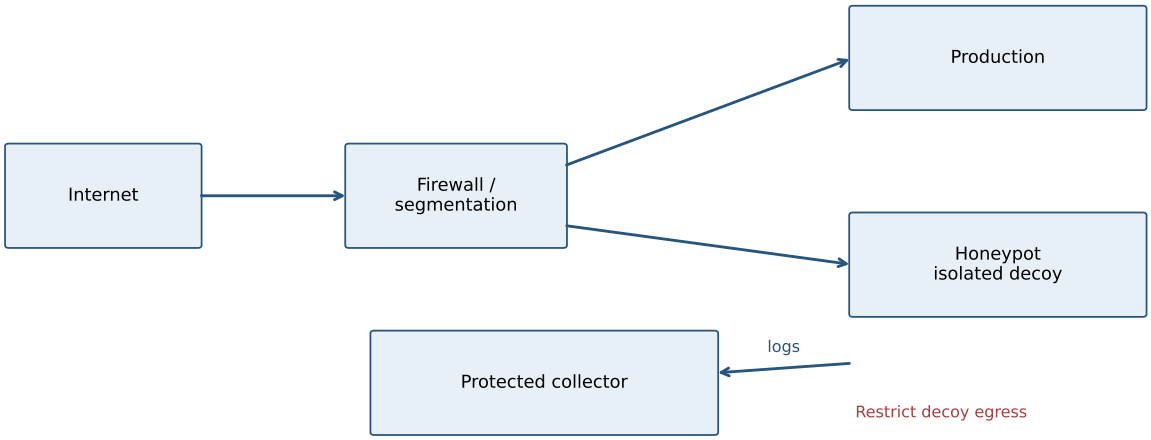
Honeypot topology and effect on normal operation · 11 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: Honeypot topology, monitoring and overhead.

> **Question summary:** Explain the need for a honeypot with topology and whether it hampers normal operation.

Exam tip. Distinguish direct production latency from operational and shared-resource cost.



A honeypot is a deliberately attractive decoy with little or no legitimate production use. Access attempts provide comparatively high-signal evidence. Interaction can reveal exploit methods, malware, commands and C&C destinations, and may divert an intruder from valuable systems.

Place it in a controlled segment/DMZ, monitor it from a protected collector and restrict its outbound reach. Never give it real credentials or a path for unrestricted attacks on production or third parties. A low-interaction decoy emulates selected services; a high-interaction host provides richer evidence at greater containment/maintenance cost.

Overhead is not zero and has no fixed percentage in the question. Costs include a VM/host, storage, logs, monitoring, analyst time, patch/reset work and some network traffic. A passive or separately segmented decoy need not sit in the production request path, so ordinary requests do not inherently incur an extra hop. Attack traffic or poor placement can nevertheless affect shared resources.

A honeypot observes attacks that touch it; it cannot replace general IDS, patching and access control.

A067 · 2021–22 · Q8(b)

OS fingerprinting to validated vulnerability finding · 12 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: Reconnaissance, fingerprinting and vulnerability assessment.

> **Question summary:** Formulate the steps for exploiting a target after scanning its OS version.

Exam tip. Separate discovery, vulnerability inference and demonstrated exploitation. Each has different evidence.

In the exam's authorized test environment:

1. Define the permitted hosts/services and obtain a baseline of expected availability.
2. Discover live hosts and open ports. A TCP connect scan completes a connection; a SYN scan interprets SYN-ACK/RST without completing an ordinary connection. Filtering/timeouts require careful interpretation.
3. Identify service banners/protocol behavior and estimate OS/version using tools such as Nmap. A banner is evidence, not proof: it can be hidden, misleading or refer to a backported patched build.
4. Match the exact product, version, configuration and platform against vendor advisories/CVE records. Check authentication requirements, exposed interfaces and patch status.
5. Validate the suspected issue safely in an isolated equivalent system, then demonstrate only the agreed impact on the test target. A version match alone does not establish successful code execution.
6. Preserve evidence, explain the root cause, patch or reconfigure, reduce exposure and retest the specific finding.

The slides mention scanners/exploit tools; a good answer connects each tool's observation to the next decision rather than treating a scan result as automatic access.

A068 · 2021–22 · Q8(c)

Address-book allowlist as a spam policy · 11 marks

Original: past-paper PDF p. 17. Sitting date: 30 October 2023. Printed Section B.

Topics: Firewall state, evasion and defense limits.

> **Question summary:** Alice rejects all email except senders in her address book. Is this an effective spam defense?

Coverage: related application/extension. Applies allowlist policy and sender-identity validation; email-specific authentication is supplementary.

Exam tip. Discuss both false negatives (spam from "allowed" identities) and false positives (new legitimate contacts).

It reduces mail from unknown addresses but is neither complete spam prevention nor an acceptable policy for every user. An address-book entry is usually an **address string**, not a cryptographic authentication of the sender. Attackers can forge visible From fields, exploit look-alike addresses or send from an actually compromised contact account.

Conversely, legitimate first-time senders, new staff, receipts and changed addresses may be rejected. That is a false-positive/usability cost and may cause users to disable the policy.

Use authenticated domain/message evidence, reputation/content controls and a quarantine/review path appropriate to the user's needs. SPF/DKIM/DMARC provide useful domain-level signals but do not prove a contact's account is uncompromised or every message benign. Strong allowlisting should use the intended authenticated identity rather than trusting a display name.

A076 · 2020–21 · Q3(b)

Web-server assessment and validation workflow · 12 marks

Original: past-paper PDF p. 18. Sitting date: 25 September 2022. Printed Section A.

Topics: Reconnaissance, fingerprinting and vulnerability assessment.

> **Question summary:** Give a stepwise approach and tools to find and exploit popular web-server vulnerabilities.

Exam tip. Separate discovery, vulnerability inference and demonstrated exploitation. Each has different evidence.

In the exam's authorized test environment:

1. Define the permitted hosts/services and obtain a baseline of expected availability.
2. Discover live hosts and open ports. A TCP connect scan completes a connection; a SYN scan interprets SYN-ACK/RST without completing an ordinary connection. Filtering/timeouts require careful interpretation.
3. Identify service banners/protocol behavior and estimate OS/version using tools such as Nmap. A banner is evidence, not proof: it can be hidden, misleading or refer to a backported patched build.
4. Match the exact product, version, configuration and platform against vendor advisories/CVE records. Check authentication requirements, exposed interfaces and patch status.
5. Validate the suspected issue safely in an isolated equivalent system, then demonstrate only the agreed impact on the test target. A version match alone does not establish successful code execution.
6. Preserve evidence, explain the root cause, patch or reconfigure, reduce exposure and retest the specific finding.

The slides mention scanners/exploit tools; a good answer connects each tool's observation to the next decision rather than treating a scan result as automatic access.

For a web server, also inventory enabled modules, TLS termination, reverse proxies and application versions; the externally visible Server header may not identify the vulnerable component. Validate the exact HTTP path/feature involved and use a reproducible minimal test with agreed limits.

A080 · 2020–21 · Q4(c)

Base-rate fallacy with a numerical example · 11 marks

Original: past-paper PDF p. 19. Sitting date: 25 September 2022. Printed Section A.

Topics: Intrusion detection, base rates and audit data.

> **Question summary:** Define the base-rate fallacy, give an example and explain its effect on daily incident handling.

Coverage: related application/extension. The deck teaches the four alarm outcomes; Bayes/base-rate calculation is an extension.

Exam tip. Use a confusion matrix or natural event counts, then compute true alarms divided by all alarms.

The fallacy is treating a detector's high sensitivity or low false-positive rate as the probability that an **alarm** is a real attack, while ignoring how rare attacks are.

Let attack prevalence $p = 0.001$, sensitivity $s = 0.99$ and false-positive rate $f = 0.01$. Bayes' rule gives

$$P(A \mid +) = \frac{sp}{sp + f(1-p)} = \frac{0.00099}{0.00099 + 0.00999} \approx 0.0902.$$

Among 100,000 events, expect 100 attacks, 99 true alarms, 99,900 benign events and 999 false alarms. Only about **9% of alarms** are true despite 99% sensitivity and 99% specificity.

Analysts face alert fatigue, wasted effort and delayed responses to real incidents. Improve precision through context/correlation, sensible thresholds, validated high-value rules and triage; reducing false positives must be balanced against false negatives. Track prevalence and evaluate precision as well as recall.

A094 · 2019–20 · Q2(c)

Verify a reported DoS source and respond with evidence · 11 marks

Original: past-paper PDF p. 22. Sitting date: 07 September 2021. Printed Section A.

Topics: Intrusion detection, base rates and audit data; OS trust, paging, boot and persistent memory images; DoS/DDoS, reflection and resource exhaustion.

> **Question summary:** A hall administrator says your shared network segment is sending DoS traffic. You deny involvement but are unsure about your roommate. How do you verify and respond?

Coverage: related application/extension. Uses IDS/network logs and OS event logging; incident-handling workflow extends the slides.

Exam tip. Explain attribution limits and the sequence verify → contain → investigate → recover.

Ask for concrete indicators: timestamps/time zone, observed source IP/port, destination, protocol, traffic rate and available logs. Confirm the report through an independent known contact path rather than trusting an unsolicited caller.

Correlate those observations with local process/network logs, router/DHCP/NAT mappings and switch ingress records. A shared public IP or segment does not uniquely identify a roommate; source IPs can also be spoofed. Capture only the relevant authorized traffic and inspect for sustained abnormal outgoing connections, malware or unexpected processes.

If evidence suggests an affected machine, coordinate containment with the administrator: isolate the specific host/port, preserve useful evidence, remove/rebuild the compromised environment, patch the entry point and rotate exposed credentials. Block abusive traffic while allowing essential investigation, then verify that it has stopped. Do not accuse an individual solely from a shared address or erase all logs before diagnosis.

A101 · 2019–20 · Q5(a)

PIDS, alarm outcomes and base-rate interpretation · 14 marks

Original: past-paper PDF p. 24. Sitting date: 07 September 2021. Printed Section B.

Topics: Intrusion detection, base rates and audit data.

> **Question summary:** Define protocol-based IDS, list alarm outcomes and explain base-rate fallacy.

Coverage: related application/extension. IDS and the confusion matrix are explicit; PIDS specialization and Bayesian analysis are supplementary.

Exam tip. Use a confusion matrix or natural event counts, then compute true alarms divided by all alarms.

A **protocol-based IDS (PIDS)** observes a protocol's messages, state transitions and expected semantics at a host/service or network boundary. It can detect invalid sequences or known malicious protocol patterns; mere TCP connectivity is not evidence that the application exchange is valid.

Actual event	Alarm	No alarm
Attack	True positive	False negative
Benign	False positive	True negative

The fallacy is treating a detector's high sensitivity or low false-positive rate as the probability that an **alarm** is a real attack, while ignoring how rare attacks are.

Let attack prevalence $p = 0.001$, sensitivity $s = 0.99$ and false-positive rate $f = 0.01$. Bayes' rule gives

$$P(A \mid +) = \frac{sp}{sp + f(1-p)} = \frac{0.00099}{0.00099 + 0.00999} \approx 0.0902.$$

Among 100,000 events, expect 100 attacks, 99 true alarms, 99,900 benign events and 999 false alarms. Only about **9% of alarms** are true despite 99% sensitivity and 99% specificity.

Analysts face alert fatigue, wasted effort and delayed responses to real incidents. Improve precision through context/correlation, sensible thresholds, validated high-value rules and triage; reducing false positives must be balanced against false negatives. Track prevalence and evaluate precision as well as recall.

A102 · 2019–20 · Q5(b)

Audit-record fields and statistical intrusion detection · 14 marks

Original: past-paper PDF p. 24. Sitting date: 07 September 2021. Printed Section B.

Topics: Intrusion detection, base rates and audit data; OS trust, paging, boot and persistent memory images.

> **Question summary:** List useful audit-record fields, explain statistical IDS and identify a weakness.

Coverage: related application/extension. Event logging and IDS sensors are taught; statistical-model design is an extension.

Exam tip. Tie the listed audit fields to the behaviors your model actually measures.

A useful event record includes timestamp/time zone, event/record ID, host/sensor, authenticated user/process and effective identity, action, target object/resource, source/destination addresses/ports as appropriate, outcome/error code and a correlation/session ID. Include enough context to link events without needlessly recording passwords or sensitive payloads.

A statistical/anomaly IDS builds a baseline or model of normal activity—such as login frequency, transfer volume, process behavior or service access. It scores new observations against that model and alerts on a threshold deviation or unlikely combination. Training, feature choice, aggregation window and threshold determine behavior.

A principal weakness is that **unusual is not identical to malicious**: legitimate new work can cause false alarms, while an attacker mimicking normal behavior may evade detection. Concept drift, compromised training data and low attack prevalence further complicate performance. Validate with labeled evidence and context, and combine anomaly scores with other signals.

A103 · 2019–20 · Q5(c)

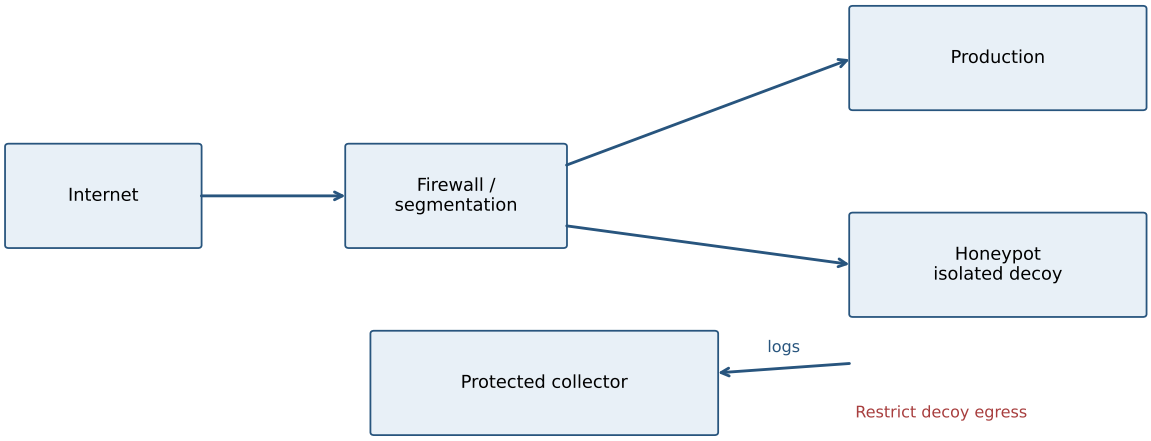
Why a decoy helps intrusion detection · 7 marks

Original: past-paper PDF p. 24. Sitting date: 07 September 2021. Printed Section B.

Topics: Honeypot topology, monitoring and overhead.

> **Question summary:** Explain why a honeypot is an effective IDS tool.

Exam tip. Distinguish direct production latency from operational and shared-resource cost.



A honeypot is a deliberately attractive decoy with little or no legitimate production use. Access attempts provide comparatively high-signal evidence. Interaction can reveal exploit methods, malware, commands and C&C destinations, and may divert an intruder from valuable systems.

Place it in a controlled segment/DMZ, monitor it from a protected collector and restrict its outbound reach. Never give it real credentials or a path for unrestricted attacks on production or third parties. A low-interaction decoy emulates selected services; a high-interaction host provides richer evidence at greater containment/maintenance cost.

Overhead is not zero and has no fixed percentage in the question. Costs include a VM/host, storage, logs, monitoring, analyst time, patch/reset work and some network traffic. A passive or separately segmented decoy need not sit in the production request path, so ordinary requests do not inherently incur an extra hop. Attack traffic or poor placement can nevertheless affect shared resources.

A honeypot observes attacks that touch it; it cannot replace general IDS, patching and access control.

A111 · 2018–19 · Q1(b)

Compare IPsec ESP and AH formats · 9 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: VPN and IPsec AH/ESP encapsulation.

Question summary: Contrast IPsec AH and ESP, including header encapsulations.

Exam tip. Mark the exact encrypted/authenticated region instead of saying simply "AH authenticates, ESP encrypts."

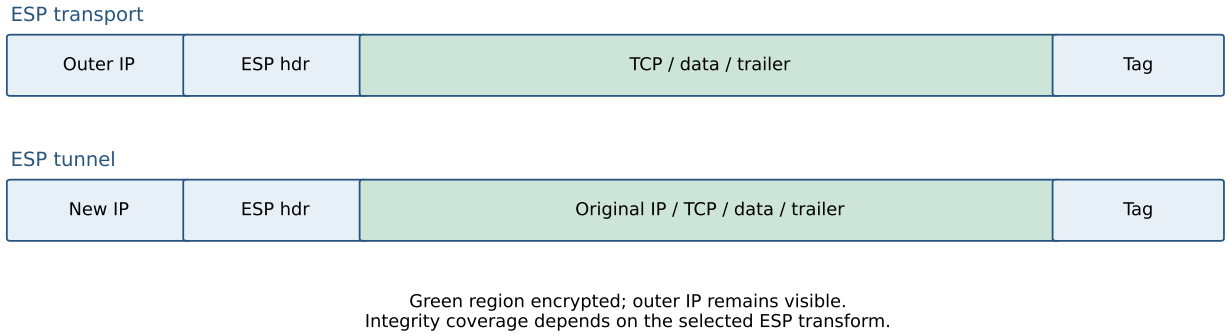
AH provides packet integrity/data-origin authentication and anti-replay when enabled, but no confidentiality. Its authentication covers payload and selected immutable/predictable IP-header fields; mutable fields cannot be protected unchanged across transit.

ESP protects the encapsulated payload with confidentiality and, when configured, integrity/authentication (normally combined in secure deployments). It does not authenticate the outer IP header in the same way as AH. Anti-replay uses sequence numbers and a receiver window, with authenticated integrity necessary to trust them.

```
AH transport: IP | AH | TCP/UDP | data
AH tunnel:    new IP | AH | original IP | TCP/UDP | data
ESP transport: IP | ESP hdr | {TCP/UDP | data | ESP trailer} | auth tag
ESP tunnel: new IP | ESP hdr | {original IP | TCP/UDP | data | trailer} | tag
```

Braces mark the ESP-encrypted part when encryption is selected. ESP header fields such as SPI/sequence number remain visible but are integrity-protected when authentication is used. Tunnel mode hides the inner addresses from ordinary transit observation while outer endpoints remain visible.

AH's protection of address fields makes ordinary NAT problematic. ESP can traverse NAT using UDP encapsulation; do not claim all IPsec is intrinsically incompatible with NAT.



References: RFC 4302, RFC 4303.

A124 · 2018–19 · Q4(c)

Rule-based ARP-spoofing detection · 9 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Intrusion detection, base rates and audit data; ARP spoofing, MITM and detection.

Question summary: Give rules a rule-based IDS can use to detect ARP spoofing.

Exam tip. Make the rule testable—state fields, trusted expectation and alert condition.

Useful rules compare observations with trusted bindings and expected behavior:

- Alert when a protected IP, especially the gateway, is claimed by an unexpected MAC.
- Alert on rapid IP-to-MAC binding changes or conflicting replies for one IP.
- Flag ARP replies inconsistent with trusted DHCP-snooping/static bindings or arriving from an unauthorized ingress port.
- Investigate excessive unsolicited/gratuitous ARP or one unexpected MAC claiming many unrelated hosts.

Correlate packet data with switch-port identity, lease records and known maintenance/failover events. A gratuitous reply or shared MAC is not automatically malicious: virtual routers, failover and proxy ARP can be legitimate. Rules need baselines/exceptions and must be evaluated for false positives/negatives.

An IDS typically alerts; DAI on a switch can enforce bindings and drop invalid packets. The same evidence can inform both, but detection and prevention are different actions.

A125 · 2018–19 · Q4(d)

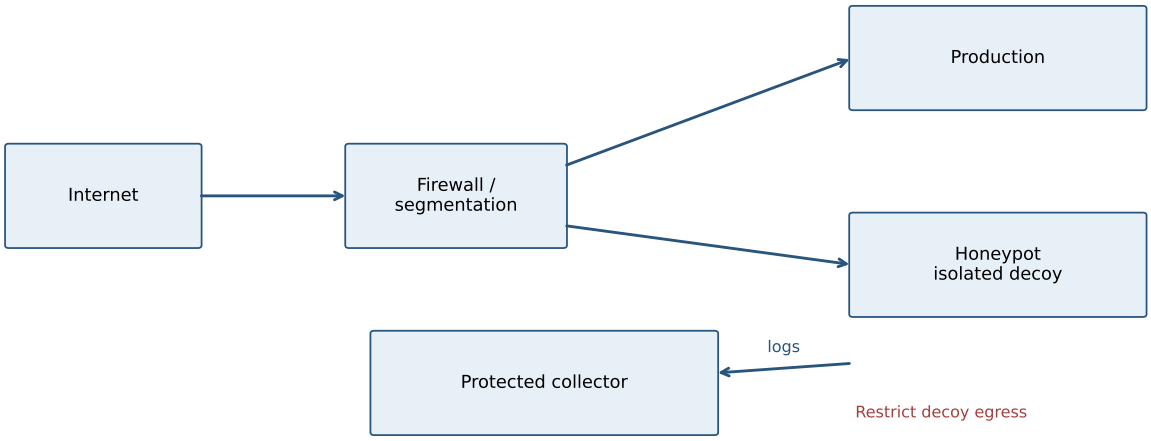
Honeypot detection, evidence and diversion · 8 marks

Original: past-paper PDF p. 26. Sitting date: 24 October 2019. Printed Section A.

Topics: Honeypot topology, monitoring and overhead.

> **Question summary:** Explain honeypot use cases with a diagram.

Exam tip. Distinguish direct production latency from operational and shared-resource cost.



A honeypot is a deliberately attractive decoy with little or no legitimate production use. Access attempts provide comparatively high-signal evidence. Interaction can reveal exploit methods, malware, commands and C&C destinations, and may divert an intruder from valuable systems.

Place it in a controlled segment/DMZ, monitor it from a protected collector and restrict its outbound reach. Never give it real credentials or a path for unrestricted attacks on production or third parties. A low-interaction decoy emulates selected services; a high-interaction host provides richer evidence at greater containment/maintenance cost.

Overhead is not zero and has no fixed percentage in the question. Costs include a VM/host, storage, logs, monitoring, analyst time, patch/reset work and some network traffic. A passive or separately segmented decoy need not sit in the production request path, so ordinary requests do not inherently incur an extra hop. Attack traffic or poor placement can nevertheless affect shared resources.

A honeypot observes attacks that touch it; it cannot replace general IDS, patching and access control.

Review checklist

- Protocol answers: label peers, message fields, authenticated identities, replay state and attacker position.
- Probability answers: define the population/conditioning, compute intermediate counts and check units.
- Memory answers: trace actual destination bounds, endianness, frame offsets and input restrictions; distinguish a control transfer from clean continuation.
- Permission answers: distinguish real/effective identity, pathname/object identity and revocation of delegated rights.
- Detection answers: discuss false positives, false negatives, attacker visibility and what the mechanism cannot observe.

Sources and section scope

Local source page maps and exam-frequency evidence are in the Sec A analysis. The Sec A syllabus defines the current topic boundary. This guide uses local Sec A answer sources and figures; it does not import Sec B answers. Older printed section labels are recorded only to locate questions, and never determine which current study guide owns a topic. Existing identifiers are preserved, so removed entries leave gaps in their numbering.

Primary technical references appear beside the answers that use them; these supplement the supplied course material rather than define a new course syllabus.